

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
ПО ПРОГРАММЕ БАКАЛАВРИАТА**

09.03.04 Программная инженерия

(код, наименование ОПОП ВО: направление подготовки, направленность (профиль))

«Разработка программно-информационных систем»

Программно-информационная система

обнаружения дефектов дорожного покрытия

(название темы)

Дипломный проект

(вид ВКР: дипломная работа или дипломный проект)

Автор ВКР

А. В. Вожов

(подпись, дата)

(инициалы, фамилия)

Группа ПО-216

Руководитель ВКР

А. Ф. Рубанов

(подпись, дата)

(инициалы, фамилия)

Нормоконтроль

А. А. Чаплыгин

(подпись, дата)

(инициалы, фамилия)

ВКР допущена к защите:

Заведующий кафедрой

А. В. Малышев

(подпись, дата)

(инициалы, фамилия)

Курск 2026 г.

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

УТВЕРЖДАЮ:

Заведующий кафедрой

(подпись, инициалы, фамилия)

«_____» _____ 20____ г.

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ ПО ПРОГРАММЕ БАКАЛАВРИАТА

Студента Вожова А. В., шифр 22-06-0127, группа ПО-21б

1. Тема «Программно-информационная система обнаружения дефектов дорожного покрытия» утверждена приказом ректора ЮЗГУ от «03» апреля 2026 г. № 1584-с.

2. Срок предоставления работы к защите «09» июня 2026 г.

3. Исходные данные для создания программной системы:

3.1. Перечень решаемых задач:

1) анализ методов обнаружения объектов с использованием компьютерного зрения и нейросетевых технологий;

2) проектирование архитектуры программно-информационной системы;

3) обучение нейросетевой модели;

4) проектирование пользовательского интерфейса системы;

5) разработка модуля обнаружения дефектов дорожного покрытия;

6) реализация механизмов сохранения результатов и формирования отчёта.

3.2. Входные данные и требуемые результаты для программы:

1) Входными данными для программной системы являются: цифровые изображения дорожного покрытия.

2) Выходными данными для программной системы являются: обработанные изображения с обнаруженными дефектами дорожного покрытия, статистическая информация и текстовый отчёт о результатах обработки.

4. Содержание работы (по разделам):

4.1. Введение.

4.1. Анализ предметной области.

4.2. Техническое задание: основание для разработки, назначение разработки, требования к программной системе, требования к пользовательскому интерфейсу, требования к оформлению документации.

4.3. Технический проект: общие сведения о программной системе, обучение нейросетевой модели, проектирование архитектуры программной системы, проектирование пользовательского интерфейса.

4.3. Технический проект:

4.4. Рабочий проект: спецификация компонентов и классов программной системы, тестирование программной системы.

4.5. Заключение.

4.6. Список использованных источников.

5. Перечень графического материала:

Лист 1. Сведения о ВКРБ.

Лист 2. Цель и задачи разработки.

Лист 3. Диаграмма прецедентов.

Лист 4. Диаграмма компонентов.

Лист 5. Интерфейс.

Лист 6. Заключение.

Руководитель ВКР

(подпись, дата)

А. Ф. Рубанов

(инициалы, фамилия)

Задание принял к исполнению

(подпись, дата)

А. В. Вожов

(инициалы, фамилия)

РЕФЕРАТ

Объем работы равен 105 страницам. Работа содержит 17 иллюстраций, 7 таблиц, 51 библиографических источников и 6 листов графического материала. Количество приложений – 2. Графический материал представлен в приложении А. Фрагменты исходного кода представлены в приложении Б.

Перечень ключевых слов: дорожное покрытие, дефекты дорожного покрытия, компьютерное зрение, нейронная сеть, YOLOv8, обнаружение объектов, обработка изображений, Python, PySide6, OpenCV, NumPy, Ultralytics, пользовательский интерфейс, инференс, обучение модели, датасет, детекция объектов, ограничивающая рамка, классификация, нейросетевой анализ, графический интерфейс.

Объектом разработки является программно-информационная система обнаружения дефектов дорожного покрытия на основе технологий компьютерного зрения и нейросетевого анализа изображений.

Целью выпускной квалификационной работы является создание программно-информационной системы обнаружения дефектов дорожного покрытия.

В процессе создания системы обнаружения дефектов дорожного покрытия были проведены анализ предметной области и существующих методов обнаружения объектов, анализ архитектуры нейросетевых моделей семейства YOLO, спроектирована архитектура программной системы и пользовательского интерфейса, разработан модуль обнаружения дефектов дорожного покрытия на основе модели YOLOv8n, выполнено обучение нейросетевой модели, реализованы механизмы загрузки, обработки, отображения и сохранения изображений, реализованы средства визуализации результатов обнаружения, формирования статистики и текстового отчёта.

При разработке программно-информационной системы использовались язык программирования Python, фреймворк PySide6, библиотеки OpenCV и NumPy, а также нейросетевая модель семейства YOLOv8.

ABSTRACT

The volume of work is 105 pages. The work contains 17 illustrations, 7 tables, 51 bibliographic sources and 6 sheets of graphic material. The number of applications is 2. The graphic material is presented in annex A. Fragments of the source code are presented in annex B.

List of keywords: road surface, pavement defects, computer vision, neural network, YOLOv8, object detection, image processing, Python, PySide6, OpenCV, NumPy, Ultralytics, user interface, inference, model learning, dataset, object detection, bounding box, classification, neural network analysis, graphical interface.

The object of the development is a software and information system for detecting pavement defects based on computer vision technologies and neural network image analysis.

In the process of creating a pavement defect detection system, an analysis of the subject area and existing object detection methods, an analysis of the architecture of neural network models of the YOLO family were carried out, the architecture of the software system and user interface was designed, a module for detecting pavement defects based on the YOLOv8n model was developed, a neural network model was trained, mechanisms for loading, processing, displaying and saving images were implemented, tools for visualizing detection results, generating statistics, and a text report are implemented.

When developing the software and information system, the Python programming language, the PySide6 framework, the OpenCV and NumPy libraries, as well as the YOLOv8 neural network model were used.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	11
1 Анализ предметной области	14
1.1 Контроль состояния дорожного покрытия	14
1.2 Методы обнаружения дефектов дорожного покрытия	15
1.2.1 Ручной визуальный контроль	15
1.2.2 Контроль с использованием специализированных диагностических комплексов	16
1.2.3 Анализ цифровых изображений	16
1.3 Методы анализа изображений	17
1.3.1 Компьютерное зрение	17
1.3.2 Свёрточные нейронные сети	18
1.3.3 Двухэтапные детекторы	19
1.3.4 Одноэтапные детекторы	19
1.3.5 Сегментационные методы анализа изображений	20
1.4 Архитектура YOLO	20
1.5 Особенности обучения нейросетевых моделей	21
1.6 Обзор существующих решений	22
1.7 Выводы по анализу предметной области	23
2 Техническое задание	24
2.1 Основание для разработки	24
2.2 Цель и назначение разработки	24
2.3 Требования к функциональности	24
2.3.1 Загрузка изображений	24
2.3.2 Настройка параметров анализа	25
2.3.3 Обработка изображений	25
2.3.4 Визуализация результатов	25
2.3.5 Сохранение и восстановление результатов	26
2.3.6 Настройка пользовательского интерфейса	26
2.4 Требования к интерфейсу	26

2.5	Моделирование вариантов использования	27
2.5.1	Вариант использования «Загрузка изображений»	29
2.5.2	Вариант использования «Настройка параметров обнаружения»	29
2.5.3	Вариант использования «Запуск анализа изображений»	29
2.5.4	Вариант использования «Просмотр результатов обнаружения»	30
2.5.5	Вариант использования «Сортировка результатов»	30
2.5.6	Вариант использования «Сохранение результатов»	31
2.5.7	Вариант использования «Формирование отчёта»	31
2.5.8	Вариант использования «Загрузка сохранённых результатов»	31
2.5.9	Вариант использования «Очистка временных данных»	32
2.5.10	Вариант использования «Переключение темы интерфейса»	32
2.6	Требования к надёжности	32
2.7	Требования к программной и технической совместимости	33
2.8	Требования к производительности	34
2.9	Требования к оформлению документации	34
3	Технический проект	35
3.1	Общая характеристика программного решения задачи	35
3.2	Обоснование выбора технологий проектирования	36
3.2.1	Язык программирования Python	36
3.2.2	Фреймворк PySide6	37
3.2.3	Библиотека OpenCV	38
3.2.4	Библиотека NumPy	38
3.2.5	Семейство нейросетевых моделей YOLOv8	39
3.2.5.1	Выбор модели семейства	39
3.3	Внедрение нейросетевой модели	41
3.3.1	Структура YOLOv8n	41
3.3.1.1	Backbone	41
3.3.1.2	Neck	41
3.3.1.3	Detection Head	41
3.3.2	Процесс обучения модели	42
3.3.3	Использование обученной модели в программной системе	45

3.4	Программная реализация системы	45
3.4.1	Главный модуль приложения	46
3.4.2	Модуль обнаружения объектов	47
3.4.3	Модуль просмотра изображений	47
3.4.4	Модуль работы с файлами	47
3.4.5	Модуль галереи изображений	48
3.4.6	Модуль управления визуальным оформлением	48
3.5	Форматы хранения и обработки данных	49
3.5.1	Входные изображения	49
3.5.2	Временные результаты	50
3.5.3	Итоговые изображения	50
3.5.4	Текстовый отчёт	50
3.6	Надёжность и производительность	51
3.6.1	Обработка ошибок	51
3.6.2	Оптимизация загрузки изображений	51
3.6.3	Управление памятью	52
3.6.4	Производительность обработки	52
3.7	Проектирование пользовательского интерфейса	53
3.8	Верификация технического проекта	54
4	Рабочий проект	56
4.1	Модуль «main.py»	56
4.2	Модуль «main_window.py»	56
4.2.1	Класс «MainWindow»	57
4.3	Модуль «detector.py»	59
4.3.1	Класс «Detector»	60
4.4	Модуль «file_manager.py»	60
4.4.1	Класс «FileManager»	61
4.5	Модуль «gallery_manager.py»	62
4.5.1	Класс «GalleryManager»	62
4.6	Модуль «viewer.py»	63
4.6.1	Класс «ImageViewer»	63

4.7	Модуль «theme_manager.py»	64
4.7.1	Класс «ThemeManager»	64
4.8	Пользовательский интерфейс программы	65
4.9	Тестирование разработанной программной системы	67
4.9.1	Модульное тестирование компонентов системы	67
4.9.2	Системное тестирование	68
	ЗАКЛЮЧЕНИЕ	79
	СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	79
	ПРИЛОЖЕНИЕ А Представление графического материала	86
	ПРИЛОЖЕНИЕ Б Фрагменты исходного кода программы	93
	На отдельных листах (CD-RW в прикрепленном конверте)	105
	Сведения о ВКРБ (Графический материал / Сведения о ВКРБ.png)	Лист 1
	Цель и задачи разработки (Графический материал / Цель и задачи разработки.png)	Лист 2
	Диаграмма прецедентов (Графический материал / Диаграмма прецедентов.png)	Лист 3
	Диаграмма компонентов (Графический материал / Диаграмма компонентов.png)	Лист 4
	Интерфейс (Графический материал / Интерфейс.png)	Лист 5
	Заключение (Графический материал / Заключение.png)	Лист 6

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

ИС – информационная система.

ПИС – программно-информационная система.

ИТ – информационные технологии.

ПО – программное обеспечение.

РП – рабочий проект.

ТЗ – техническое задание.

ТП – технический проект.

ЯП – язык программирования.

ИИ – искусственный интеллект.

НС – нейронная сеть.

СВ – сверточная вычислительная сеть.

CV – компьютерное зрение (Computer Vision).

GUI – графический пользовательский интерфейс (Graphical User Interface).

API – интерфейс прикладного программирования (Application Programming Interface).

CNN – сверточная нейронная сеть (Convolutional Neural Network).

ML – машинное обучение (Machine Learning).

DL – глубокое обучение (Deep Learning).

IoU – коэффициент пересечения областей (Intersection over Union).

FPS – количество кадров в секунду (Frames Per Second).

GPU – графический процессор (Graphics Processing Unit).

CPU – центральный процессор (Central Processing Unit).

YOLOv8n – модель обнаружения объектов семейства YOLO восьмого поколения.

ВВЕДЕНИЕ

Современное развитие транспортной инфраструктуры требует постоянного контроля состояния дорожного покрытия и своевременного выявления возникающих дефектов. Рост протяжённости автомобильных дорог и интенсивности движения приводит к увеличению объёмов диагностических работ, что делает задачу автоматизации анализа дорожного полотна особенно актуальной.

Традиционные методы обследования дорог в значительной степени основаны на ручном анализе и визуальном контроле. Подобные подходы требуют существенных временных затрат и обладают ограниченной эффективностью при обработке больших объёмов данных. В связи с этим всё более широкое распространение получают программно-информационные системы, использующие методы компьютерного зрения и машинного обучения для автоматического анализа изображений.

Современные нейросетевые модели обнаружения объектов позволяют выполнять обработку изображений с высокой скоростью и точностью. Использование таких технологий делает возможным создание прикладных программных систем, способных автоматически выявлять дефекты дорожного покрытия на цифровых изображениях и формировать результаты анализа в удобной для пользователя форме.

Разработка подобных систем представляет практический интерес как для организаций, осуществляющих мониторинг дорожной инфраструктуры, так и для исследовательских задач в области компьютерного зрения.

Цель настоящей работы – создание программно-информационной системы обнаружения дефектов дорожного покрытия на основе нейросетевых методов анализа изображений. Для достижения поставленной цели необходимо решить *следующие задачи*:

- анализ предметной области и существующих методов обнаружения дефектов дорожного покрытия;

- изучение особенностей архитектур нейросетевых моделей обнаружения объектов;
- проектирование программной системы;
- обучение нейросетевой модели;
- проектирование пользовательского интерфейса приложения;
- реализация обработки изображений с использованием нейросетевой модели;
- реализация механизмов сохранения результатов и формирования отчёта.

Структура и объем работы. Отчет состоит из введения, 4 разделов основной части, заключения, списка использованных источников, 2 приложений. Текст выпускной квалификационной работы равен 105 страницам.

Во введении обоснована актуальность темы выпускной квалификационной работы, сформулированы цель и задачи разработки, приведена структура работы.

В первом разделе выполнен анализ предметной области, рассмотрены существующие подходы к обнаружению дефектов дорожного покрытия, методы компьютерного зрения и архитектуры нейросетевых моделей.

Во втором разделе представлено техническое задание на разработку программно-информационной системы, сформулированы требования к функциональным возможностям, интерфейсу.

В третьем разделе описан технический проект системы, приведено обоснование выбора технологий разработки, рассмотрена архитектура программной системы и особенности внедрения нейросетевой модели.

В четвертом разделе представлено описание программной реализации, приведены сведения о тестировании и проверке работоспособности разработанной системы.

В заключении приведены основные результаты работы и выводы по итогам разработки программно-информационной системы обнаружения дефектов дорожного покрытия.

В приложении А представлен графический материал. В приложении Б представлены фрагменты исходного кода программы.

1 Анализ предметной области

1.1 Контроль состояния дорожного покрытия

Состояние дорожного покрытия является одним из ключевых факторов, влияющих на безопасность дорожного движения, комфорт эксплуатации транспортных средств и эффективность транспортной инфраструктуры в целом. Нарушения целостности покрытия приводят к увеличению аварийности, ускоренному износу автомобилей, повышению затрат на ремонт и ухудшению транспортной доступности.

В процессе укладки и эксплуатации дорожное покрытие подвергается постоянному воздействию различных факторов, среди которых:

- интенсивная транспортная нагрузка;
- температурные перепады;
- воздействие атмосферных осадков;
- циклы замерзания и оттаивания;
- естественное старение материалов;
- нарушение технологии строительства и ремонта.

Под дефектами дорожного покрытия понимаются локальные или распределённые нарушения структуры полотна, возникающие вследствие механических нагрузок, климатических воздействий, ошибок строительства, износа материалов и недостаточного обслуживания.

К наиболее распространённым дефектам относятся:

- выбоины;
- продольные и поперечные трещины;
- сеточные разрушения;
- колеиность;
- просадки;
- шелушение покрытия;
- разрушение кромок дорожного полотна.

Своевременное обнаружение дефектов позволяет предотвращать дальнейшее разрушение покрытия, снижать вероятность дорожно-транспортных

происшествий, оптимизировать планирование ремонтных работ, а также сокращать затраты на содержание дорожной инфраструктуры.

Традиционный контроль состояния дорожного полотна осуществляется посредством визуального обследования специалистами. Такой подход обладает рядом недостатков:

- высокая трудоёмкость;
- значительные временные затраты;
- зависимость от квалификации инспектора;
- субъективность оценки;
- сложность регулярного мониторинга больших территорий.

В связи с этим актуальной задачей является автоматизация процессов диагностики дорожного покрытия с использованием современных методов обработки изображений и машинного обучения.

1.2 Методы обнаружения дефектов дорожного покрытия

Существующие методы анализа состояния дорожного полотна можно разделить на несколько основных групп.

1.2.1 Ручной визуальный контроль

Данный метод предполагает проведение обследования дорожного покрытия специалистами непосредственно на участке дороги.

Преимущества метода являются:

- высокая точность при участии квалифицированного эксперта;
- возможность комплексной оценки состояния покрытия;
- отсутствие необходимости в сложном оборудовании.

Недостатки метода:

- низкая скорость обследования;
- высокая стоимость при больших объёмах;
- человеческий фактор;
- невозможность непрерывного мониторинга.

1.2.2 Контроль с использованием специализированных диагностических комплексов

Данный подход основан на применении мобильных лабораторий, оснащённых:

- лазерными сканерами;
- стереокамерами;
- профилографами;
- датчиками вибрации;
- GPS/ГЛОНАСС системами.

Такие комплексы обеспечивают высокую точность измерений, однако обладают существенными недостатками:

- высокая стоимость оборудования;
- сложность эксплуатации;
- ограниченная доступность для организаций.

1.2.3 Анализ цифровых изображений

Метод основан на обработке фотографий дорожного покрытия с целью выявления характерных признаков дефектов.

Источниками изображений могут выступать:

- стационарные камеры;
- мобильные устройства;
- автомобильные видеорегистраторы;
- беспилотные летательные аппараты;
- дорожные инспекционные комплексы.

Преимущества метода:

- низкая стоимость внедрения;
- возможность масштабирования;
- автоматизация анализа;
- возможность интеграции в существующие системы мониторинга.

1.3 Методы анализа изображений

1.3.1 Компьютерное зрение

Компьютерное зрение представляет собой направление искусственного интеллекта, связанное с разработкой методов автоматического анализа и интерпретации визуальной информации. Основная цель данной области заключается в создании систем, способных извлекать значимые сведения из изображений и видеопотока аналогично тому, как это делает человек.

Формирование компьютерного зрения как самостоятельной научной дисциплины началось в 1960–1970-х годах. Исследования были ориентированы на решение сравнительно простых задач: выделение контуров объектов, анализ геометрических форм и распознавание символов. Для анализа использовались алгоритмы фильтрации, бинаризации, выделения границ и морфологических преобразований. Среди наиболее известных подходов данного периода можно выделить оператор Собеля, оператор Кэнни и преобразование Хафа.

Постепенно компьютерное зрение развивалось благодаря росту вычислительных мощностей и появлению методов анализа текстур, статистического распознавания образов и первых алгоритмов машинного обучения.

Основными задачами компьютерного зрения являются:

- классификация изображений;
- сегментация;
- обнаружение объектов;
- отслеживание объектов;
- анализ пространственных характеристик сцены.

Тем не менее классические методы имеют существенные ограничения. Они требуют ручного проектирования признаков и плохо адаптируются к изменению условий съёмки, освещения, масштаба объектов и наличию шумов.

1.3.2 Свёрточные нейронные сети

Значительный этап развития компьютерного зрения связан с появлением искусственных нейронных сетей, а особенно — свёрточных нейронных сетей (Convolutional Neural Networks, CNN). Их развитие стало возможным благодаря увеличению вычислительных мощностей графических процессоров и накоплению больших размеченных наборов изображений.

Первые идеи использования локальных свёрток для анализа изображений были предложены ещё в 1980-х годах. Существенный прорыв произошёл в 1998 году с появлением архитектуры LeNet, разработанной Яном Лекуном для распознавания рукописных цифр. Действительно широкое распространение свёрточные сети получили после публикации модели AlexNet в 2012 году, которая продемонстрировала значительное превосходство над классическими методами на конкурсе ImageNet.

Основное отличие свёрточных сетей от традиционных алгоритмов заключается в способности автоматически извлекать признаки из изображений: CNN самостоятельно обучается находить информативные признаки на разных уровнях абстракции.

Архитектура свёрточной нейронной сети обычно включает последовательность свёрточных слоёв, функций активации, слоёв подвыборки и полносвязных слоёв. На начальных этапах обработки сеть выявляет простые элементы изображения — линии, углы, контуры. Более глубокие слои формируют представления сложных структур и объектов.

Для задач анализа дорожного покрытия использование свёрточных нейронных сетей является особенно эффективным, поскольку дефекты асфальтового полотна обладают сложной визуальной структурой и могут значительно отличаться по форме, размеру и условиям съёмки.

Основная цель нейросетевых алгоритмов обнаружения состоит в определении:

- принадлежности объекта к определённому классу;
- координат ограничивающей рамки объекта;

- вероятности корректности предсказания.

Современные алгоритмы делятся на две основные категории.

1.3.3 Двухэтапные детекторы

К ним относятся семейства:

- R-CNN;
- Fast R-CNN;
- Faster R-CNN;
- Mask R-CNN.

Принцип работы включает генерацию кандидатов областей и классификацию и уточнение координат.

Преимущества:

- высокая точность;
- хорошее качество локализации.

Недостатки:

- высокая вычислительная сложность;
- низкая скорость обработки;
- ограниченная пригодность для работы в реальном времени.

1.3.4 Одноэтапные детекторы

К данной категории относятся:

- SSD;
- RetinaNet;
- семейство YOLO.

Они выполняют обнаружение объектов за один проход нейронной сети.

Преимущества:

- высокая скорость;
- возможность работы в режиме реального времени;
- хорошее соотношение точности и производительности.

Для задачи обнаружения дефектов дорожного покрытия одноэтапные детекторы являются наиболее предпочтительными.

1.3.5 Сегментационные методы анализа изображений

Помимо алгоритмов обнаружения объектов, в компьютерном зрении активно применяются методы семантической и instance-сегментации.

Семантическая сегментация предполагает классификацию каждого пикселя изображения, что позволяет получить точное выделение областей, относящихся к определённым классам. Для решения подобных задач широко используются архитектуры:

- U-Net;
- SegNet;
- DeepLab;
- PSPNet.

Преимуществами сегментационных моделей являются:

- высокая точность пространственной локализации;
- возможность анализа сложных геометрических структур;
- получение детализированных масок объектов;
- высокая информативность результатов.

Однако применение таких методов связано с рядом ограничений.

Во-первых, для обучения требуется детальная разметка данных, при которой необходимо выделять контуры объектов на уровне пикселей. Подготовка подобных наборов данных требует значительных временных затрат.

Во-вторых, сегментационные модели предъявляют повышенные требования к вычислительным ресурсам и, как правило, уступают детекторам объектов по скорости обработки.

В задачах, где приоритетом является оперативность анализа и возможность работы в режиме реального времени, более рациональным является использование детекторов объектов.

1.4 Архитектура YOLO

Семейство алгоритмов YOLO (You Only Look Once) является одним из наиболее известных решений в области обнаружения объектов на теку-

щий момент. Первая версия модели была представлена Джозефом Редмоном в 2015 году и стала значимым этапом развития компьютерного зрения.

Главная идея YOLO заключается в отказе от многоэтапной обработки изображения. Вместо последовательного поиска областей интереса и последующей классификации сеть рассматривает изображение как единую задачу регрессии. Изображение разбивается на сетку, для каждой ячейки которой вычисляются параметры ограничивающих рамок и вероятности присутствия объектов. Такой подход существенно ускоряет обработку по сравнению с двухэтапными детекторами.

Основными преимуществами архитектуры являются:

- высокая скорость работы;
- компактность моделей, и, как следствие, возможность применения на пользовательских устройствах;
- удобство обучения на пользовательских наборах данных;
- высокая точность обнаружения при должном обучении.

1.5 Особенности обучения нейросетевых моделей

Эффективность современных систем компьютерного зрения напрямую зависит от качества обучающих данных.

Процесс подготовки набора данных представляет собой один из наиболее трудоёмких этапов разработки и включает:

- сбор исходных изображений;
- разметку объектов;
- проверку корректности аннотаций;
- разделение выборки на обучающую, валидационную и тестовую;
- предварительную обработку данных.

Одним из важнейших этапов является аугментация данных, позволяющая искусственно расширить обучающую выборку и повысить устойчивость модели к различным условиям.

К основным методам аугментации относятся:

- изменение яркости;

- изменение контрастности;
- поворот изображения;
- масштабирование;
- зеркальное отражение;
- случайное кадрирование;
- добавление шумов;
- размытие;
- изменение цветового баланса.

Использование аугментации позволяет моделировать реальные условия эксплуатации системы и снижает вероятность переобучения.

Существенной проблемой также является дисбаланс классов, при котором отдельные категории объектов представлены в выборке значительно чаще других. В подобных условиях требуется дополнительная балансировка обучающих данных.

Качество обучающего набора оказывает непосредственное влияние на точность и устойчивость итоговой модели.

1.6 Обзор существующих решений

В России идёт развитие различных интеллектуальных систем мониторинга дорожного покрытия.

Одним из наиболее известных отечественных решений является система «СКАНДОР», предназначенная для автоматизированного выявления дефектов дорожного покрытия и элементов дорожной инфраструктуры с применением технологий искусственного интеллекта. Система ориентирована на использование в составе интеллектуальных транспортных систем и предусматривает интеграцию с муниципальными службами.

Несмотря на высокий технологический уровень подобных решений, большинство существующих систем ориентированы на промышленное или государственное применение и обладают рядом ограничений: высокой стоимостью внедрения, необходимостью специализированного оборудования и закрытостью архитектуры.

1.7 Выводы по анализу предметной области

Проведённый анализ показал, что традиционные методы контроля состояния дорожного покрытия не обеспечивают необходимой оперативности и масштабируемости.

Наиболее перспективным направлением является использование методов, основанных на свёрточных нейронных сетях.

Среди современных архитектур обнаружения объектов оптимальным решением для поставленной задачи являются модели семейства YOLO, обеспечивающие баланс между точностью распознавания и скоростью обработки.

2 Техническое задание

2.1 Основание для разработки

Основанием для разработки является задание на выпускную квалификационную работу бакалавра на тему «Программно-информационная система обнаружения дефектов дорожного покрытия».

2.2 Цель и назначение разработки

Целью разработки является создание программно-информационной системы, предназначенной для автоматизированного обнаружения дефектов дорожного покрытия на основе анализа цифровых изображений с использованием методов компьютерного зрения и нейросетевых алгоритмов.

Назначением системы является повышение скорости выявления дефектов дорожного полотна, а также снижение трудозатрат при визуальном контроле состояния автомобильных дорог.

Основными задачами разработки являются:

- анализ существующих методов обнаружения дефектов дорожного покрытия;
- выбор алгоритма детекции объектов;
- разработка программной архитектуры системы;
- реализация модуля обработки изображений;
- обучение и интеграция модели нейросети;
- разработка графического пользовательского интерфейса;
- реализация средств визуализации результатов;
- тестирование системы.

2.3 Требования к функциональности

2.3.1 Загрузка изображений

Система должна обеспечивать:

- выбор одного или нескольких файлов изображений;

- предварительную проверку поддерживаемого формата;
- отображение миниатюр загруженных изображений;
- отображение информации о количестве загруженных изображений.

Загрузка должна осуществляться посредством стандартного диалогового окна операционной системы.

2.3.2 Настройка параметров анализа

Система должна предоставлять пользователю возможность настройки параметров обнаружения дефектов, включая:

- изменение порога достоверности обнаружения;
- выбор директории временного хранения;
- выбор режима сортировки обработанных изображений.

2.3.3 Обработка изображений

Подсистема анализа должна обеспечивать:

- применение обученной нейросетевой модели;
- последовательную обработку загруженных изображений;
- обнаружение объектов, относящихся к дефектам покрытия;
- формирование координат ограничивающих рамок;
- вычисление вероятности принадлежности объекта к классу.

Во время обработки пользователь должен получать информацию о текущем прогрессе выполнения.

2.3.4 Визуализация результатов

Система должна обеспечивать:

- отображение обработанных изображений;
- визуальное выделение найденных дефектов;
- вывод статистической информации;
- отображение уровня достоверности обнаружения.

2.3.5 Сохранение и восстановление результатов

Система должна поддерживать:

- сохранение обработанных изображений;
- формирование текстового отчёта;
- загрузку сохранённых результатов;
- возможность сортировки вновь открытых результатов.

Также должно быть возможно:

- очистить временные данные результатов анализа;
- выбрать каталог для хранения временных результатов.

2.3.6 Настройка пользовательского интерфейса

Система должна предоставлять:

- переключение между светлой и тёмной темами;
- автоматическое обновление визуального оформления при изменении размера окна;
- сохранение выбранной темы между сеансами работы.

2.4 Требования к интерфейсу

Интерфейс программно-информационной системы должен представлять собой графическое настольное приложение с интуитивно понятным пользовательским управлением.

Основные элементы интерфейса должны включать:

- главное окно приложения;
- область отображения загруженных изображений;
- область отображения результатов анализа;
- меню управления обработкой;
- меню выбора файлов, сохранения и загрузки результатов;
- меню изменения вида главного окна и порядка выведенных результатов;
- панель статистики.

2.5 Моделирование вариантов использования

В системе выделяется одно действующее лицо: оператор системы — выполняет загрузку изображений, запуск анализа и просмотр результатов обработки.

Перечень основных вариантов использования:

1. Загрузка изображений.
2. Настройка параметров обнаружения.
3. Запуск анализа изображений.
4. Просмотр результатов обнаружения.
5. Сортировка результатов.
6. Сохранение результатов.
7. Формирование отчёта.
8. Загрузка ранее сохранённых результатов.
9. Очистка временных данных.
10. Переключение темы интерфейса.

Диаграмма прецедентов представлена на рисунке 2.1.

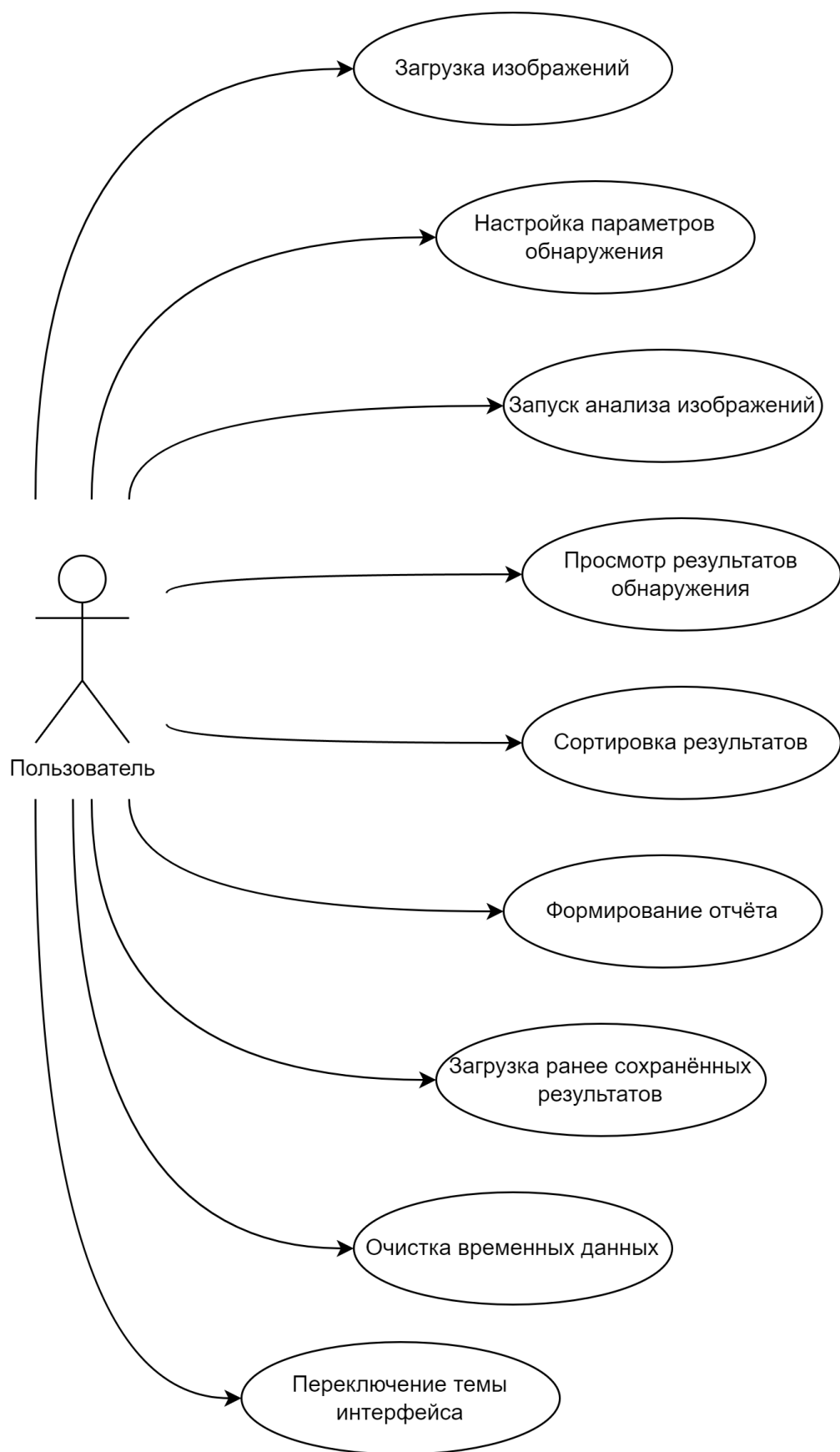


Рисунок 2.1 – Диаграмма прецедентов для пользователя

2.5.1 Вариант использования «Загрузка изображений»

Заинтересованные лица и их требования: оператор желает загрузить одно или несколько изображений дорожного покрытия для последующего анализа.

Предусловие: приложение запущено, отображается зона вывода входных изображений.

Постусловие: выбранные изображения отображаются в интерфейсе системы.

Основной успешный сценарий:

1. Оператор нажимает кнопку загрузки изображений.
2. Система открывает диалог выбора файлов.
3. Оператор выбирает один или несколько файлов.
4. Изображения по ходу загрузки отображаются в зоне вывода входных изображений.

2.5.2 Вариант использования «Настройка параметров обнаружения»

Заинтересованные лица и их требования: оператор желает настроить параметры анализа для повышения точности обнаружения.

Предусловие: изображения загружены.

Постусловие: новые параметры сохранены и готовы к использованию.

Основной успешный сценарий:

1. Оператор открывает меню обработки.
2. Во вкладке выбора порога обнаружения изменяет значение.
3. Система сохраняет настройки.

2.5.3 Вариант использования «Запуск анализа изображений»

Заинтересованные лица и их требования: оператор желает автоматически обнаружить дефекты дорожного покрытия.

Предусловие: изображения загружены.

Постусловие: изображения обработаны, результаты сохранены во временной папке.

Основной успешный сценарий:

1. Оператор открывает меню обработки и нажимает кнопку запуска анализа.
2. Выполняется обработка изображений, прогресс выводится в строке статистики.
3. Результаты отображаются в интерфейсе.

2.5.4 Вариант использования «Просмотр результатов обнаружения»

Заинтересованные лица и их требования: оператор желает визуально оценить результаты анализа определенного изображения.

Предусловие: результаты обработки выведены на экран.

Постусловие: оператор получает доступ к результатам обработки.

Основной успешный сценарий:

1. Оператор выбирает обработанное изображение.
2. Система отображает изображение с выделенными дефектами.
3. Отображается количество найденных объектов и вероятности их обнаружения.

2.5.5 Вариант использования «Сортировка результатов»

Заинтересованные лица и их требования: оператор желает упорядочить результаты для удобства анализа.

Предусловие: результаты обработки выведены на экран.

Постусловие: результаты отсортированы по выбранному критерию.

Основной успешный сценарий:

1. Оператор в меню вида выбирает критерий сортировки.
2. Система выполняет перестроение списка.
3. Отображается обновлённый порядок результатов.

2.5.6 Вариант использования «Сохранение результатов»

Заинтересованные лица и их требования: оператор желает сохранить результаты анализа.

Предусловие: анализ завершён.

Постусловие: результаты записаны в выбранный каталог.

Основной успешный сценарий:

1. Оператор открывает меню и выбирает команду сохранения.
2. В открывшемся диалоговом окне указывает каталог.
3. Система сохраняет обработанные изображения и сопутствующие данные.

2.5.7 Вариант использования «Формирование отчёта»

Заинтересованные лица и их требования: оператор желает получить текстовый отчёт по результатам анализа.

Предусловие: анализ завершён.

Постусловие: сформирован текстовый файл отчёта.

Основной успешный сценарий:

1. Оператор открывает меню и выбирает команду сохранения.
2. В открывшемся диалоговом окне указывает каталог.
3. Система сохраняет обработанные изображения и формирует текстовый отчёт.

2.5.8 Вариант использования «Загрузка сохранённых результатов»

Заинтересованные лица и их требования: оператор желает повторно открыть ранее сохранённые результаты.

Предусловие: существуют сохранённые данные, файл отчета не поврежден и не изменен.

Постусловие: результаты отображаются в интерфейсе.

Основной успешный сценарий:

1. Оператор в меню выбирает открытие обработанных данных.
2. В открывшемся диалоговом окне указывает каталог.
3. Система считывает данные.
4. Результаты отображаются.

2.5.9 Вариант использования «Очистка временных данных»

Заинтересованные лица и их требования: оператор желает удалить временные файлы.

Предусловие: во временном каталоге имеются данные.

Постусловие: временные файлы удалены.

Основной успешный сценарий:

1. Оператор в меню обработки выбирает очистку временных данных.
2. Система запрашивает подтверждение.
3. После подтверждения выполняется удаление.

2.5.10 Вариант использования «Переключение темы интерфейса»

Заинтересованные лица и их требования: оператор желает изменить внешний вид интерфейса.

Предусловие: приложение активно.

Постусловие: применяется новая цветовая тема.

Основной успешный сценарий:

1. Оператор открывает меню вида.
2. Выбирает тему оформления.
3. Система применяет изменения.

2.6 Требования к надёжности

Разрабатываемая программно-информационная система должна обеспечивать стабильную работу при обработке изображений, корректно реагировать на ошибки ввода и предотвращать потерю временных результатов.

Устойчивость обработки изображений. Система должна корректно обрабатывать ошибки, возникающие при загрузке файлов неподдерживаемого формата, повреждённых изображений.

Контроль выполнения анализа. При запуске процедуры обнаружения дефектов система должна обеспечивать завершение обработки каждого изображения независимо от ошибок, возникших при анализе других файлов.

Сохранность результатов. Результаты обработки должны сохраняться во временном каталоге до завершения пользовательской сессии либо до их явного удаления пользователем.

2.7 Требования к программной и технической совместимости

Система должна корректно функционировать на персональных компьютерах, удовлетворяющих минимальным требованиям к программному обеспечению и вычислительным ресурсам.

Операционная система. Приложение должно обеспечивать корректную работу в среде операционных систем семейства Windows.

Используемые библиотеки. Для функционирования системы требуется наличие следующих программных библиотек:

- PySide6 — для реализации графического пользовательского интерфейса;
- OpenCV — для предварительной обработки изображений;
- NumPy — для выполнения численных операций;
- Ultralytics YOLO — для выполнения детекции дефектов;

Аппаратные требования. Минимальные технические характеристики вычислительного устройства:

- процессор с тактовой частотой не менее 2 ГГц;
- оперативная память не менее 8 ГБ;
- свободное место на диске не менее 4 ГБ;
- графический адаптер (видеокарта).

Форматы входных данных. Система должна поддерживать обработку изображений форматов JPG, JPEG и PNG.

Совместимость модели. Для корректной работы требуется наличие файла обученной модели нейронной сети формата .pt.

2.8 Требования к производительности

Система должна обеспечивать приемлемое время отклика пользовательского интерфейса и достаточную скорость обработки изображений.

Время загрузки приложения не должно превышать 10 секунд.

Время обработки одного изображения размером до 1920x1080 пикселей не должно превышать 5 секунд при использовании рекомендуемой аппаратной конфигурации.

Отзывчивость интерфейса. Во время выполнения обработки система должна отображать индикатор выполнения и предотвращать выполнение конфликтующих операций.

2.9 Требования к оформлению документации

Разработка программной документации и программного изделия должна производиться согласно ГОСТ 19.102-77 и ГОСТ 34.601-90. Единая система программной документации.

3 Технический проект

3.1 Общая характеристика программного решения задачи

Разработанная программно-информационная система обнаружения дефектов дорожного покрытия представляет собой настольное приложение, предназначенное для автоматизированного анализа изображений дорожного полотна с использованием методов компьютерного зрения и нейросетевой обработки данных.

Основной задачей системы является выявление дефектов дорожного покрытия на входных изображениях, их локализация и визуальное отображение результатов для последующего анализа пользователем.

Программное решение реализовано в виде локального приложения, не требующего постоянного сетевого соединения. Все этапы обработки данных выполняются на стороне пользователя, что обеспечивает автономность работы системы и снижает зависимость от внешних сервисов.

Обработка изображений осуществляется последовательно: входные данные загружаются пользователем через графический интерфейс, после чего передаются в модуль нейросетевой детекции. Результаты анализа сохраняются и отображаются в виде графической и текстовой информации.

Архитектура системы ориентирована на модульный принцип построения, при котором функциональность разделена на независимые компоненты, взаимодействующие через центральный управляющий модуль. Такой подход обеспечивает расширяемость системы, упрощает сопровождение программного кода и позволяет в дальнейшем добавлять новые функциональные возможности без изменения существующей логики.

Особое внимание при проектировании системы уделено обеспечению высокой скорости обработки изображений, рациональному использованию вычислительных ресурсов и корректному управлению графическим интерфейсом пользователя.

3.2 Обоснование выбора технологий проектирования

При разработке программно-информационной системы обнаружения дефектов дорожного покрытия выбор технологического стека осуществлялся исходя из требований к производительности, удобству разработки, возможности интеграции современных методов компьютерного зрения, а также обеспечения удобного графического пользовательского интерфейса.

В результате анализа существующих инструментальных средств был выбран технологический стек, включающий язык программирования Python, графический фреймворк PySide6, библиотеку OpenCV, нейросетевую модель семейства YOLOv8 и библиотеку NumPy.

3.2.1 Язык программирования Python

В качестве основного языка разработки программной системы был выбран Python.

Выбор данного языка обусловлен спецификой решаемой задачи. Разрабатываемая система сочетает в себе графический пользовательский интерфейс, обработку изображений и применение нейросетевой модели обнаружения объектов. Python предоставляет единое программное окружение для реализации всех перечисленных компонентов без необходимости использования нескольких языков программирования.

Существенным преимуществом Python является наличие развитой экосистемы библиотек для компьютерного зрения и машинного обучения. Это позволило использовать готовые инструменты для работы с изображениями, выполнения нейросетевого анализа и построения графического интерфейса приложения.

Дополнительным преимуществом языка является высокая скорость разработки и простота модификации программного кода. Это особенно важно при экспериментальной настройке параметров обработки изображений и тестировании различных вариантов работы модели обнаружения.

Следует отметить, что Python уступает компилируемым языкам программирования по скорости выполнения отдельных вычислительных операций. Однако в рамках данного проекта этот недостаток не оказывает существенного влияния на производительность системы, поскольку основные ресурсоёмкие вычисления выполняются средствами библиотеки Ultralytics и задействуют оптимизированные низкоуровневые реализации.

3.2.2 Фреймворк PySide6

Для разработки графического интерфейса выбран фреймворк PySide6, представляющий собой официальный набор Python-привязок для библиотеки Qt.

При выборе альтернатив рассматривались следующие решения:

1. Tkinter — отличается простотой использования, однако обладает ограниченными возможностями построения современных интерфейсов.
2. Kivy — ориентирован преимущественно на мультитач-интерфейсы и мобильные приложения.

Выбор PySide6 обусловлен его ориентацией на создание полнофункциональных настольных приложений и тесной интеграцией с языком Python. Фреймворк предоставляет развитый набор визуальных компонентов, систему обработки событий и средства управления состоянием интерфейса.

К основным возможностям PySide6 относятся:

- создание оконных приложений со сложной иерархией элементов;
- событийно-ориентированная модель взаимодействия;
- механизм сигналов и слотов;
- встроенные средства хранения пользовательских настроек;
- поддержка стилизации интерфейса;
- кроссплатформенность.

Особое значение для разрабатываемой системы имеет механизм сигналов и слотов, обеспечивающий удобное взаимодействие между независимыми компонентами приложения без жёсткой связанности между ними.

Дополнительным преимуществом PySide6 является наличие большого числа готовых интерфейсных компонентов, что сокращает объём низкоуровневой реализации и упрощает сопровождение программного кода.

3.2.3 Библиотека OpenCV

Для выполнения операций компьютерного зрения была использована библиотека OpenCV.

OpenCV является одной из наиболее распространённых библиотек для обработки изображений и предоставляет широкий набор инструментов для анализа графических данных.

В рамках разработанной системы библиотека используется для решения следующих задач:

- преобразование форматов изображений;
- построение ограничивающих рамок;
- нанесение текстовых меток;
- сохранение обработанных изображений;
- цветовые преобразования;
- подготовка изображений к последующей визуализации.

Выбор OpenCV обусловлен следующими преимуществами:

- высокой производительностью;
- оптимизацией под работу с большими изображениями;
- поддержкой множества форматов данных;
- эффективной интеграцией с NumPy;
- совместимостью с нейросетевыми моделями.

3.2.4 Библиотека NumPy

Для эффективной работы с массивами данных и преобразования изображений использовалась библиотека NumPy.

Данная библиотека обеспечивает высокопроизводительные операции над многомерными массивами и широко применяется в задачах обработки изображений и машинного обучения.

В разработанной системе NumPy используется для:

- преобразования буферов изображений;
- подготовки данных для нейросетевой модели;
- изменения структуры массивов;
- оптимизации операций обработки изображений.

Использование NumPy обеспечивает эффективное взаимодействие между компонентами PySide6, OpenCV и нейросетевой моделью.

3.2.5 Семейство нейросетевых моделей YOLOv8

В качестве алгоритма обнаружения дефектов дорожного покрытия была выбрано семейство моделей YOLOv8.

Подход YOLO основан на одноэтапном обнаружении объектов, она выполняет локализацию и классификацию одновременно за один проход.

В рамках проекта использовалась облегчённая версия модели, обеспечивающая снижение вычислительных затрат при сохранении приемлемого качества обнаружения.

Для адаптации модели к задаче обнаружения дефектов дорожного покрытия было выполнено её дополнительное обучение на специализированном датасете изображений дорожного полотна.

3.2.5.1 Выбор модели семейства

При выборе архитектуры нейросетевой модели учитывались следующие требования:

- высокая скорость обработки изображений;
- достаточная точность обнаружения дефектов;
- умеренные требования к вычислительным ресурсам;
- простота интеграции в Python-приложение.

В рамках семейства YOLOv8 доступно несколько моделей:

- YOLOv8n (nano);
- YOLOv8s (small);
- YOLOv8m (medium);

- YOLOv8l (large);
- YOLOv8x (extra large).

Для настоящего проекта была выбрана облегчённая версия YOLOv8n. Это обеспечивает существенное сокращение времени обработки.

Архитектурно модель включает следующие основные компоненты:

- backbone-сеть для извлечения признаков;
- neck-блок для объединения признаков различных масштабов;
- detection head для локализации объектов и определения классов.

Выбор данной модификации обусловлен следующими причинами.

1. Модель обладает минимальным количеством параметров, что снижает требования к вычислительным ресурсам.

2. YOLOv8n обеспечивает высокую скорость инференса, что особенно важно при пакетной обработке большого количества изображений дорожного покрытия.

3. Облегчённая архитектура позволяет использовать систему на персональных компьютерах средней производительности без необходимости применения специализированных GPU-решений.

Несмотря на компактность, YOLOv8n демонстрирует достаточную точность для задач обнаружения локальных дефектов дорожного покрытия при условии корректной подготовки обучающей выборки.

Дополнительным преимуществом выбора YOLOv8 является наличие библиотеки Ultralytics, предоставляющей удобный интерфейс для:

- обучения модели;
- выполнения инференса;
- сохранения весов;
- настройки параметров обнаружения;
- интеграции в пользовательские приложения.

3.3 Внедрение нейросетевой модели

3.3.1 Структура YOLOv8n

Структурно модель состоит из трёх основных блоков: Backbone, Neck и Detection Head. Её упрощенная схема представлена на рисунке 3.1.

3.3.1.1 Backbone

Backbone (на схеме — область со слоями) построен на модифицированной CSPDarknet. В нём используются блоки c2f (CSPNet с двумя свёртками, объединяющий выходы нескольких внутренних блоков для лучшего распространения информации) и модуль SPPF (быстрое пирамидальное объединение через max pooling, расширяющее рецептивное поле). Задача backbone — извлечь из входного изображения ($640 \times 640 \times 3$) карты признаков, описывающие текстуру, границы и форму дефектов.

3.3.1.2 Neck

Neck реализован как PAN-FPN (Path Aggregation Network + Feature Pyramid Network). На схеме он представлен блоком «Feature fusion». Сюда поступают признаки от разных слоёв backbone — сплошными стрелками передаются глубокие семантические признаки, пунктирными — детальные пространственные признаки. Neck объединяет их с помощью повышения разрешения (upsampling) и конкатенации. Это формирует многомасштабное представление, необходимое для обнаружения как мелких трещин, так и крупных выбоин.

3.3.1.3 Detection Head

Голова (Head) на схеме разделена на две ветви: «Classification» (классификация — определение типа дефекта) и «Regression» (регрессия — уточнение координат ограничивающей рамки). Такая архитектура называется decoupled head (раздельная голова). Модель является anchor-free (не использует заранее заданные якоря), что упрощает обучение. Для повышения точ-

ности локализации применяется Distribution Focal Loss (DFL) — функция потерь, которая предсказывает распределение вероятностей для границ бокса, а не единственное значение. На выходе модель формирует три тензора для масштабов 80×80 (мелкие объекты), 40×40 (средние) и 20×20 (крупные).

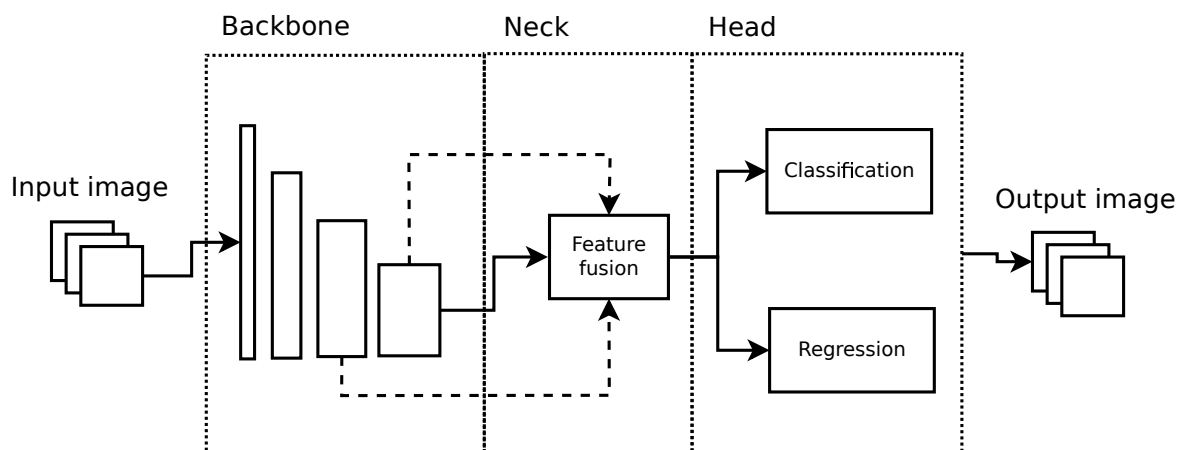


Рисунок 3.1 – Упрощенная схема структуры модели нейросети

3.3.2 Процесс обучения модели

Эффективность работы системы автоматического обнаружения дефектов дорожного покрытия напрямую зависит от качества обучения нейросетевой модели. Для адаптации выбранной архитектуры к предметной области был выполнен этап дообучения предварительно обученной модели на специализированном наборе изображений дорожного покрытия.

В качестве исходной модели использовалась предварительно обученная версия YOLOv8n, содержащая веса, полученные на крупномасштабном датасете общего назначения.

Использование предварительно обученной модели позволяет существенно сократить время обучения и повысить итоговую точность обнаружения за счёт использования уже сформированных низкоуровневых признаков представлений.

Данный подход особенно эффективен для задач, где объём специализированной обучающей выборки ограничен.

Процесс обучения выполнялся с использованием отдельного программного модуля `train_model.py`.

Инициализация модели осуществлялась следующим образом:

```
1 model = YOLO("models/yolov8n.pt")
```

На данном этапе происходила загрузка базовой архитектуры YOLOv8n с предобученными весами.

После инициализации запускался процесс обучения:

```
1 model.train(  
2 data="dataset/pothole_dataset/data.yaml",  
3 epochs=50,  
4 batch=16,  
5 device=0  
6 )
```

Обучение выполнялось на специализированном датасете дорожных дефектов, содержащем размеченные изображения участков дорожного покрытия.

Каждое изображение в обучающей выборке сопровождалось аннотацией, содержащей координаты ограничивающих прямоугольников, локализирующих области дефектов.

Структура набора данных соответствовала формату YOLO для обеспечения совместимости датасета с инструментами библиотеки Ultralytics и включала следующие компоненты:

- каталог обучающих изображений;
- каталог валидационных изображений;
- файлы аннотаций;
- конфигурационный файл `data.yaml`.

Файл конфигурации содержал информацию о расположении выборок, количестве классов и их наименованиях.

Процесс обучения включал несколько последовательных этапов.

На первом этапе выполнялась загрузка обучающих данных и их предварительная обработка.

Предварительная обработка включала:

- нормализацию изображений;

- масштабирование;
- преобразование координат аннотаций;
- формирование обучающих батчей.

На втором этапе происходило прямое распространение сигнала через нейронную сеть.

В процессе прямого прохода модель формировала прогнозы относительно наличия дефектов, их координат и уровня достоверности обнаружения.

На третьем этапе вычислялась функция потерь, включающая несколько составляющих:

- ошибку локализации;
- ошибку классификации;
- ошибку оценки достоверности.

Полученное значение ошибки использовалось для обратного распространения градиента и корректировки весовых коэффициентов сети.

Данный процесс повторялся для всех батчей текущей эпохи.

После завершения каждой эпохи выполнялась валидация модели на отдельной выборке.

Валидация позволяла контролировать степень обобщающей способности модели и предотвращать переобучение.

В ходе обучения автоматически сохранялись контрольные точки модели.

Наиболее эффективная версия весов сохранялась в файл:

<code>runs/detect/train/weights/best.pt</code>
--

Именно данный файл впоследствии использовался в основной программной системе для выполнения обнаружения дефектов.

Использование отдельного файла весов обеспечивает разделение этапов обучения и эксплуатации модели.

Это позволяет:

- переобучать модель независимо от основной системы;
- использовать различные версии модели;

– при необходимости быстро выполнять замену весов.

Применение такого подхода повышает гибкость программной системы и упрощает её дальнейшую модернизацию.

В результате обучения была получена специализированная модель, адаптированная к задаче обнаружения дефектов дорожного покрытия и пригодная для интеграции в пользовательское приложение.

3.3.3 Использование обученной модели в программной системе

После завершения процесса обучения полученная нейросетевая модель была интегрирована в разработанную программно-информационную систему для выполнения автоматического обнаружения дефектов дорожного покрытия на пользовательских изображениях.

Интеграция модели реализована посредством подключения файла весовых коэффициентов, полученных в процессе обучения.

Загрузка обученной модели выполняется при инициализации основного окна приложения:

```
1 self.detector = Detector(  
2     YOLO("runs/detect/train/weights/best.pt"),  
3     conf_threshold=0.25  
4 )
```

3.4 Программная реализация системы

Система обнаружения дефектов дорожного покрытия построена на основе модульной архитектуры, в которой каждый компонент выполняет строго определённую функциональную роль. Все модули взаимодействуют через центральный управляющий класс, обеспечивающий координацию процессов обработки изображений и работы пользовательского интерфейса.

Система включает в себя следующие модули:

1. Главный модуль приложения: обеспечивает координацию всех подсистем и управление состоянием интерфейса.
2. Модуль обнаружения объектов: реализует обработку изображений с использованием нейросетевой модели YOLOv8.

3. Модуль просмотра изображений: обеспечивает просмотр входных и обработанных изображений в отдельном окне.

4. Модуль работы с файлами: отвечает за выбор, сохранение и загрузку данных, а также формирование отчётов.

5. Модуль галереи результатов: реализует отображение результатов обработки в виде сеточной структуры.

6. Модуль управления визуальным оформлением: обеспечивает переключение между светлой и тёмной темой оформления.

На рисунке 3.2 представлена диаграмма компонентов проектируемой системы.

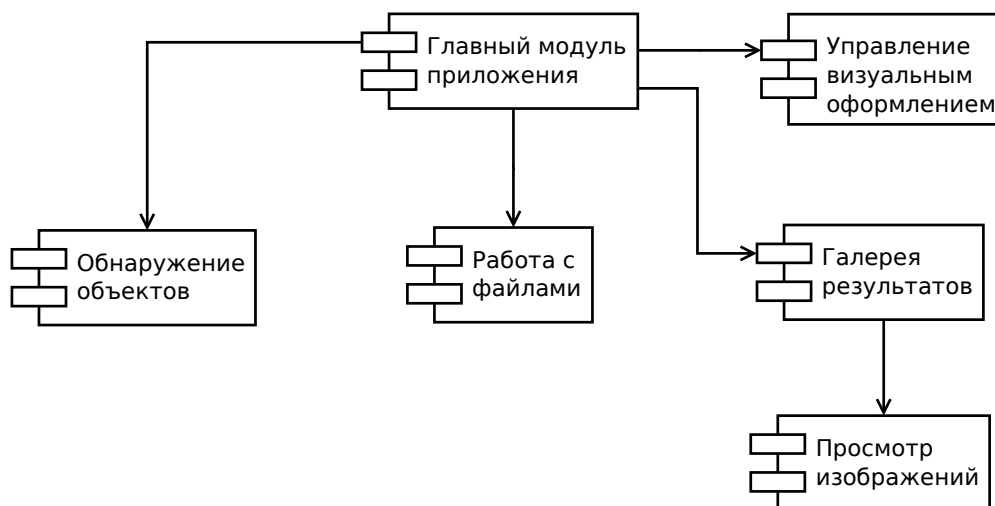


Рисунок 3.2 – Диаграмма компонентов проектируемой системы

3.4.1 Главный модуль приложения

Главный модуль является центральным управляющим компонентом системы. Он обеспечивает инициализацию всех подсистем и координацию их взаимодействия.

В задачи главного модуля входят:

- инициализация графического интерфейса пользователя;
- создание вспомогательных модулей системы;
- управление состоянием приложения;
- запуск процесса обработки изображений;

- отображение статистики работы системы;
- обработка пользовательских событий.

Главный модуль агрегирует все функциональные компоненты системы и обеспечивает их согласованную работу в рамках единого приложения.

3.4.2 Модуль обнаружения объектов

Модуль обнаружения объектов предназначен для обработки входных изображений с использованием обученной нейросетевой модели YOLOv8n.

Модуль выполняет последовательную обработку изображений, включающую подготовку данных, выполнение нейросетевого анализа и визуализацию результатов. В процессе работы осуществляется преобразование изображений в формат, совместимый с моделью, после чего выполняется инференс и получение координат обнаруженных объектов.

Результаты детекции визуализируются в виде ограничивающих прямоугольников с указанием класса объекта и уровня уверенности модели. Для оптимизации производительности предусмотрено ограничение максимального размера обрабатываемых изображений.

3.4.3 Модуль просмотра изображений

Модуль просмотра изображений предназначен для последовательного просмотра отдельных изображений в увеличенном масштабе.

Модуль обеспечивает:

- отображение изображения в адаптивном масштабе;
- навигацию между изображениями;
- автоматическое масштабирование под размер окна.

Данный компонент позволяет пользователю детально анализировать результаты работы системы.

3.4.4 Модуль работы с файлами

Модуль работы с файлами обеспечивает взаимодействие программы с файловой структурой операционной системы.

Основные функции модуля включают:

- выбор входных изображений;
- сохранение результатов обработки;
- формирование текстового отчёта;
- загрузку ранее сохранённых результатов;
- управление временными файлами обработки.

В процессе сохранения данных формируется отчёт, содержащий информацию о количестве обработанных изображений, числе обнаруженных объектов, среднем количестве детекций на изображение и времени обработки.

3.4.5 Модуль галереи изображений

Модуль галереи обеспечивает визуальное представление как исходных, так и обработанных изображений, а также отвечает за перестроение интерфейса при изменении состояния приложения.

Основные функции модуля:

- отображение миниатюр входных изображений;
- формирование миниатюр обработанных изображений;
- размещение элементов в сетке;
- отображение количества обнаруженных объектов для каждого изображения;
- адаптацию интерфейса к изменению размеров окна.

Галерея изображений обеспечивает удобный визуальный анализ входных и обработанных данных.

3.4.6 Модуль управления визуальным оформлением

Модуль управления темой отвечает за изменение визуального оформления пользовательского интерфейса.

Система поддерживает два режима отображения: светлый и тёмный. Выбранная тема сохраняется в настройках приложения и автоматически применяется при следующем запуске системы.

Модуль изменяет стилизацию основных элементов интерфейса, включая окна, кнопки, панели и элементы управления.

3.5 Форматы хранения и обработки данных

В процессе функционирования программно-информационной системы используются несколько категорий данных, обеспечивающих полный цикл обработки изображений дорожного покрытия: входные изображения, временные результаты обработки, итоговые обработанные изображения и текстовый отчёт.

Организация хранения данных построена таким образом, чтобы обеспечить удобство доступа, минимизацию избыточности и возможность последующего анализа результатов обнаружения дефектов.

3.5.1 Входные изображения

Входными данными системы являются цифровые изображения дорожного покрытия, выбираемые пользователем через графический интерфейс программы.

Поддерживаются следующие графические форматы:

- JPEG (*.jpg, *.jpeg);
- PNG (*.png);
- BMP (*.bmp).

Выбор данных форматов обусловлен их широкой распространённостью и поддержкой библиотек Qt и OpenCV.

При загрузке изображений осуществляется автоматическая проверка корректности формата и возможность чтения файла. Для корректного отображения ориентации применяется автоматическое преобразование метаданных изображения посредством метода `setAutoTransform()`.

При необходимости изображения масштабируются до допустимого размера без нарушения пропорций, что позволяет снизить вычислительную нагрузку при обработке.

3.5.2 Временные результаты

Промежуточные результаты работы сохраняются во временную директорию, определяемую пользователем либо устанавливаемую по умолчанию.

Временная папка содержит изображения после обработки нейронной сетью.

При запуске новой обработки временная директория автоматически пересоздаётся, что исключает накопление устаревших результатов.

Путь к директории сохраняется в пользовательских настройках с использованием механизма QSettings.

3.5.3 Итоговые изображения

Итоговые результаты сохраняются в формате JPEG.

Каждое обработанное изображение содержит:

- исходное изображение дорожного покрытия;
- прямоугольные ограничивающие рамки обнаруженных дефектов;
- подпись класса;
- числовое значение достоверности обнаружения.

Имена файлов формируются автоматически по шаблону result_XXXX.jpg, где XXXX — порядковый номер изображения.

3.5.4 Текстовый отчёт

По завершении обработки формируется текстовый отчёт в формате TXT.

Отчёт содержит:

- количество загруженных изображений;
- количество успешно обработанных изображений;
- суммарное число обнаруженных дефектов;
- среднее количество дефектов на изображение;
- общее время обработки;
- детализацию по каждому изображению.

Структура отчёта обеспечивает возможность последующего документирования результатов и использования данных в аналитических целях. Также он используется системой для восстановления статистической информации о ранее обработанных изображениях при их повторной загрузке.

3.6 Надёжность и производительность

Одним из ключевых требований к разрабатываемой системе является обеспечение стабильной работы при обработке больших наборов изображений, а также эффективное использование вычислительных ресурсов при выполнении детекции.

Для достижения требуемого уровня надёжности и производительности в системе реализован комплекс решений, связанных с обработкой исключительных ситуаций, оптимизацией загрузки данных и рациональным управлением памятью.

3.6.1 Обработка ошибок

Система предусматривает обработку основных исключительных ситуаций, возникающих при работе с файлами, пользовательским вводом и нейросетевой обработкой.

К основным контролируемым ошибкам относятся:

- попытка открытия неподдерживаемого файла;
- невозможность чтения изображения;
- отсутствие доступа к директории сохранения;
- повреждение файлов результатов;
- прерывание обработки пользователем.

Пользователь уведомляется о возникших проблемах посредством диалоговых окон и сообщений в строке состояния.

3.6.2 Оптимизация загрузки изображений

Для повышения скорости работы применяется предварительное масштабирование изображений на этапе чтения.

Использование класса QImageReader позволяет загружать изображения сразу в уменьшенном разрешении, что существенно снижает:

- время декодирования;
- объём используемой оперативной памяти;
- нагрузку на процессор.

Дополнительно реализована отложенная перестройка сетки результатов посредством таймера, что предотвращает избыточные операции перерасчёта интерфейса.

3.6.3 Управление памятью

Рациональное использование памяти достигается за счёт:

- последовательной обработки изображений;
- удаления устаревших временных объектов;
- использования масштабированных копий вместо полноразмерных изображений;
- применения механизма deleteLater() для корректного освобождения ресурсов Qt.

При отображении миниатюр используются только уменьшенные копии изображений, что позволяет сохранять отзывчивость интерфейса.

3.6.4 Производительность обработки

Основное время вычислений приходится на выполнение инференса модели YOLOv8.

Общая скорость обработки зависит от:

- разрешения входных изображений;
- количества изображений;
- вычислительной мощности GPU;
- выбранного порога обнаружения.

Для контроля производительности реализован замер общего времени обработки.

Полученные показатели отображаются пользователю в интерфейсе программы и фиксируются в отчёте.

3.7 Проектирование пользовательского интерфейса

Проектирование пользовательского интерфейса разработанной системы осуществлялось с учётом требований, представленных в пункте 2.4 технического задания.

Интерфейс системы реализован на основе графического фреймворка PySide6 и представляет собой настольное приложение с событийно-ориентированной моделью взаимодействия.

Интерфейс программно-информационной системы включает следующие основные элементы:

- главное окно приложения;
- меню управления обработкой изображений;
- область отображения входных изображений;
- область отображения результатов обработки;
- окно расширенного просмотра изображений;
- панель статистики обработки;
- меню управления файловыми операциями;
- меню изменения вида главного окна и сортировки результатов.

На рисунке 3.3 представлен макет пользовательского интерфейса главного окна программной системы.



Рисунок 3.3 – Макет пользовательского интерфейса главного окна

3.8 Верификация технического проекта

Проверка работы программно-информационной системы должна включать:

1. Запуск приложения и корректную инициализацию главного окна.
2. Загрузку изображений поддерживаемых форматов.
3. Проверку обработки изображений нейросетевой моделью YOLOv8n.
4. Корректное отображение исходных и обработанных изображений в галереях.
5. Открытие изображений в режиме полноразмерного просмотра.
6. Навигацию между изображениями в модуле просмотра.
7. Сохранение обработанных изображений в выбранную директорию.
8. Формирование текстового отчёта по результатам обработки.
9. Загрузку ранее сохранённых результатов.
10. Переключение между светлой и тёмной темами оформления.
11. Проверку сохранения пользовательских настроек после перезапуска приложения.

12. Обработку изображений различного разрешения.
13. Корректную очистку временных файлов при повторной обработке.
14. Устойчивую работу системы при последовательной обработке большого количества изображений.
15. Корректное завершение работы приложения с освобождением используемых ресурсов.

4 Рабочий проект

Программно-информационная система обнаружения дефектов дорожного покрытия включает несколько программных модулей: модуль запуска приложения, главный модуль приложения, модуль обнаружения дефектов, модуль работы с файлами, модуль галереи изображений, модуль просмотра изображений, модуль управления визуальным оформлением.

4.1 Модуль «main.py»

Модуль «main.py» является точкой входа в программно-информационную систему обнаружения дефектов дорожного покрытия. Основной задачей модуля является создание экземпляра приложения Qt, инициализация главного окна программы и запуск цикла обработки событий пользовательского интерфейса.

Функции модуля запуска приложения представлены в таблице 4.1.

Таблица 4.1 – Описание функции, используемой в модуле запуска приложения

Название функции	Аргументы	Описание функции
1	2	3
main	Аргументов нет	Точка входа в приложение. Выполняет создание объекта класса QApplication, создание экземпляра класса MainWindow, отображение главного окна программы и запуск основного цикла обработки событий пользовательского интерфейса.

4.2 Модуль «main_window.py»

Модуль «main_window.py» содержит класс «MainWindow», реализующий главное окно программно-информационной системы обнаружения дефектов дорожного покрытия. Класс обеспечивает взаимодействие пользователя с системой, управление обработкой изображений, отображение резуль-

татов обнаружения дефектов, настройку параметров работы приложения и сохранение результатов обработки.

4.2.1 Класс «MainWindow»

Методы класса «MainWindow» представлены в таблице 4.2.

Таблица 4.2 – Описание методов класса «MainWindow»

Название метода	Аргументы	Описание метода
1	2	3
<code>__init__</code>	Аргументов нет	Конструктор класса. Выполняет инициализацию главного окна приложения, создание необходимых объектов и настройку пользовательского интерфейса.
<code>init_window</code>	Аргументов нет	Настройка основных параметров главного окна приложения, включая заголовок и размеры окна.
<code>init_settings</code>	Аргументов нет	Загрузка пользовательских настроек приложения и настройка каталога для хранения временных результатов обработки.
<code>init_state</code>	Аргументов нет	Инициализация внутренних структур данных и переменных состояния приложения.
<code>init_managers</code>	Аргументов нет	Создание экземпляров классов <code>ThemeManager</code> , <code>FileManager</code> и <code>GalleryManager</code> .
<code>init_detector</code>	Аргументов нет	Создание экземпляра класса <code>Detector</code> и загрузка обученной модели обнаружения дефектов дорожного покрытия.

Продолжение таблицы 4.2

1	2	3
init_menu	Аргументов нет	Формирование главного меню приложения и привязка действий пользователя к соответствующим обработчикам.
init_wid	Аргументов нет	Создание и настройка элементов пользовательского интерфейса главного окна приложения.
init_statusbar	Аргументов нет	Создание строки состояния и элементов отображения статистики обработки изображений.
update_progress	current, total, text	Обновление индикатора выполнения операции и отображение текущего состояния процесса.
show_welcome_screen	Аргументов нет	Отображение стартового экрана приложения с краткой инструкцией по работе с системой.
clear_welcome_screen	Аргументов нет	Удаление стартового экрана из рабочей области приложения.
update_results_dir_display	Аргументов нет	Обновление отображаемого пути к каталогу временных результатов обработки.
set_images	files	Сохранение списка выбранных изображений и запуск формирования миниатюр входных данных.
save_results	Аргументов нет	Сохранение обработанных изображений и отчёта в выбранный пользователем каталог.
open_processed_results	Аргументов нет	Загрузка ранее сохранённых результатов обработки изображений.

Продолжение таблицы 4.2

1	2	3
toggle_input_images	checked	Управление отображением панели входных изображений.
set_conf_threshold	value	Установка порога уверенности обнаружения объектов нейронной сетью.
sort_results	mode	Сортировка результатов обработки по имени файла или количеству обнаруженных объектов.
resizeEvent	event	Обработка изменения размеров окна приложения и перестроение области отображения результатов.
select_image	index	Выбор изображения для последующего просмотра в отдельном окне.
open_viewer	index	Открытие окна просмотра исходных изображений.
open_processed_viewer	index	Открытие окна просмотра обработанных изображений.
detect_objects	Аргументов нет	Выполнение обнаружения дефектов дорожного покрытия на выбранных изображениях с использованием нейросетевой модели.
closeEvent	event	Обработка завершения работы приложения и контроль сохранения результатов обработки.

4.3 Модуль «detector.py»

Модуль «detector.py» содержит класс «Detector», реализующий обнаружение дефектов дорожного покрытия на изображениях с использованием предварительно обученной нейросетевой модели. Модуль обеспечивает за-

грузку изображений, подготовку данных для обработки, запуск нейросетевого анализа, визуализацию результатов обнаружения и сохранение обработанных изображений.

4.3.1 Класс «Detector»

Методы класса «Detector» представлены в таблице 4.3.

Таблица 4.3 – Описание методов класса «Detector»

Название метода	Аргументы	Описание метода
1	2	3
<code>__init__</code>	<code>model</code> , <code>conf_threshold</code> , <code>max_dim</code>	Конструктор класса. Выполняет сохранение экземпляра нейросетевой модели, установку порога обнаружения объектов и максимального размера обрабатываемого изображения.
<code>set_conf</code>	<code>value</code>	Установка нового значения порога уверенности обнаружения объектов.
<code>process_images</code>	<code>images</code> , <code>results_dir</code> , <code>progress_callback</code> , <code>stop_flag</code>	Выполнение обнаружения дефектов дорожного покрытия на наборе изображений. Выполняет загрузку и масштабирование изображений, запуск нейросетевой модели, подсчёт обнаруженных объектов, визуализацию результатов обнаружения и сохранение обработанных изображений. Возвращает список обработанных файлов, количество обнаруженных объектов на каждом изображении и общее число обнаруженных объектов.

4.4 Модуль «file_manager.py»

Модуль «file_manager.py» содержит класс «FileManager», предназначенный для выполнения операций с файлами и каталогами. Модуль обеспечивает загрузку исходных изображений, сохранение результатов обработки,

формирование текстового отчёта, управление временными файлами, а также загрузку ранее сохранённых результатов обработки.

4.4.1 Класс «FileManager»

Методы класса «FileManager» представлены в таблице 4.4.

Таблица 4.4 – Описание методов класса «FileManager»

Название метода	Аргументы	Описание метода
1	2	3
<code>__init__</code>	<code>window</code>	Конструктор класса. Сохраняет ссылку на главное окно приложения для последующего взаимодействия с его элементами интерфейса.
<code>open_images</code>	Аргументов нет	Открытие диалогового окна выбора изображений и передача выбранных файлов в главное окно приложения.
<code>save_results</code>	Аргументов нет	Сохранение обработанных изображений в выбранный каталог. Выполняет проверку существования файлов, обработку конфликтов имён и формирование отчёта по результатам обработки.
<code>create_report</code>	<code>folder</code>	Создание текстового отчёта о выполненной обработке изображений в каталоге <code>folder</code> . В отчёт включаются статистические данные и информация по каждому обработанному изображению.
<code>select_results_folder</code>	Аргументов нет	Выбор каталога для хранения временных результатов обработки и сохранение выбранного пути в настройках приложения.
<code>clear_results_folder</code>	Аргументов нет	Очистка каталога временных результатов обработки после подтверждения пользователем.

Продолжение таблицы 4.4

1	2	3
open_processed_results	Аргументов нет	Открытие ранее сохранённых результатов обработки. Выполняет выбор каталога и загрузку информации из отчёта.
_load_report	folder, report_path	Чтение и анализ файла отчёта, расположенного по пути report_path. Формирует структуру данных, содержащую сведения о результатах обработки изображений.

4.5 Модуль «gallery_manager.py»

Модуль «gallery_manager.py» содержит класс «GalleryManager», предназначенный для отображения изображений в пользовательском интерфейсе программы. Модуль обеспечивает формирование галереи исходных изображений, отображение результатов обнаружения дефектов дорожного покрытия, обновление сетки результатов при изменении размеров окна и управление отображением панели входных изображений.

4.5.1 Класс «GalleryManager»

Методы класса «GalleryManager» представлены в таблице 4.5.

Таблица 4.5 – Описание методов класса «GalleryManager»

Название метода	Аргументы	Описание метода
1	2	3
__init__	window	Конструктор класса. Сохраняет ссылку на главное окно приложения для взаимодействия с элементами интерфейса.

Продолжение таблицы 4.5

1	2	3
update_result_grid	Аргументов нет	Формирование и обновление сетки обработанных изображений. Выполняет размещение миниатюр результатов обработки и информации о количестве обнаруженных объектов.
display_results	Аргументов нет	Загрузка обработанных изображений, создание их уменьшенных копий и подготовка данных для отображения в галерее результатов.
show_thumbnails	Аргументов нет	Загрузка и отображение миниатюр исходных изображений в левой панели интерфейса. Обеспечивает отображение индикатора выполнения процесса загрузки.
clear_layout	layout	Очистка элементов пользовательского интерфейса, расположенных в контейнере layout.
toggle_input_visibility	checked	Управление отображением панели исходных изображений в зависимости от значения параметра checked.

4.6 Модуль «viewer.py»

Модуль «viewer.py» содержит класс «ImageViewer», предназначенный для просмотра изображений в отдельном окне. Модуль обеспечивает отображение выбранного изображения, масштабирование изображения при изменении размеров окна и навигацию между изображениями.

4.6.1 Класс «ImageViewer»

Методы класса «ImageViewer» представлены в таблице 4.6.

Таблица 4.6 – Описание методов класса «ImageViewer»

Название метода	Аргументы	Описание метода
1	2	3
<code>__init__</code>	<code>images, index</code>	Конструктор класса. Выполняет создание окна просмотра изображений, настройку элементов интерфейса и отображение изображения с номером <code>index</code> из списка <code>images</code> .
<code>show_image</code>	Аргументов нет	Загрузка и отображение текущего изображения с автоматическим масштабированием под размеры окна просмотра.
<code>resizeEvent</code>	<code>event</code>	Обработчик изменения размеров окна. Выполняет повторное масштабирование и отображение изображения после изменения размеров окна просмотра.
<code>prev_image</code>	Аргументов нет	Переход к предыдущему изображению в списке изображений.
<code>next_image</code>	Аргументов нет	Переход к следующему изображению в списке изображений.

4.7 Модуль «theme_manager.py»

Модуль «theme_manager.py» содержит класс «ThemeManager», предназначенный для управления визуальным оформлением пользовательского интерфейса. Модуль обеспечивает переключение между светлой и тёмной темами оформления и сохранение выбранных параметров интерфейса.

4.7.1 Класс «ThemeManager»

Методы класса «ThemeManager» представлены в таблице 4.7.

Таблица 4.7 – Описание методов класса «ThemeManager»

Название метода	Аргументы	Описание метода
1	2	3
<code>__init__</code>	<code>window</code>	Конструктор класса. Сохраняет ссылку на главное окно приложения.
<code>set_theme</code>	<code>theme</code>	Установка темы оформления интерфейса в соответствии со значением параметра <code>theme</code> . Выполняет сохранение выбранной темы в настройках приложения.
<code>apply_dark_theme</code>	Аргументов нет	Применение тёмной темы оформления пользовательского интерфейса.
<code>apply_light_theme</code>	Аргументов нет	Применение светлой темы оформления пользовательского интерфейса.

4.8 Пользовательский интерфейс программы

Пользовательский интерфейс программно-информационной системы обнаружения дефектов дорожного покрытия реализован с использованием библиотеки PySide6 и построен по принципу графического оконного приложения.

После запуска программы пользователю отображается стартовый экран с краткой инструкцией по работе с системой и кнопками быстрого доступа к основным функциям (рисунок 4.1). Для начала работы необходимо выбрать изображения для анализа с помощью команды открытия файлов.

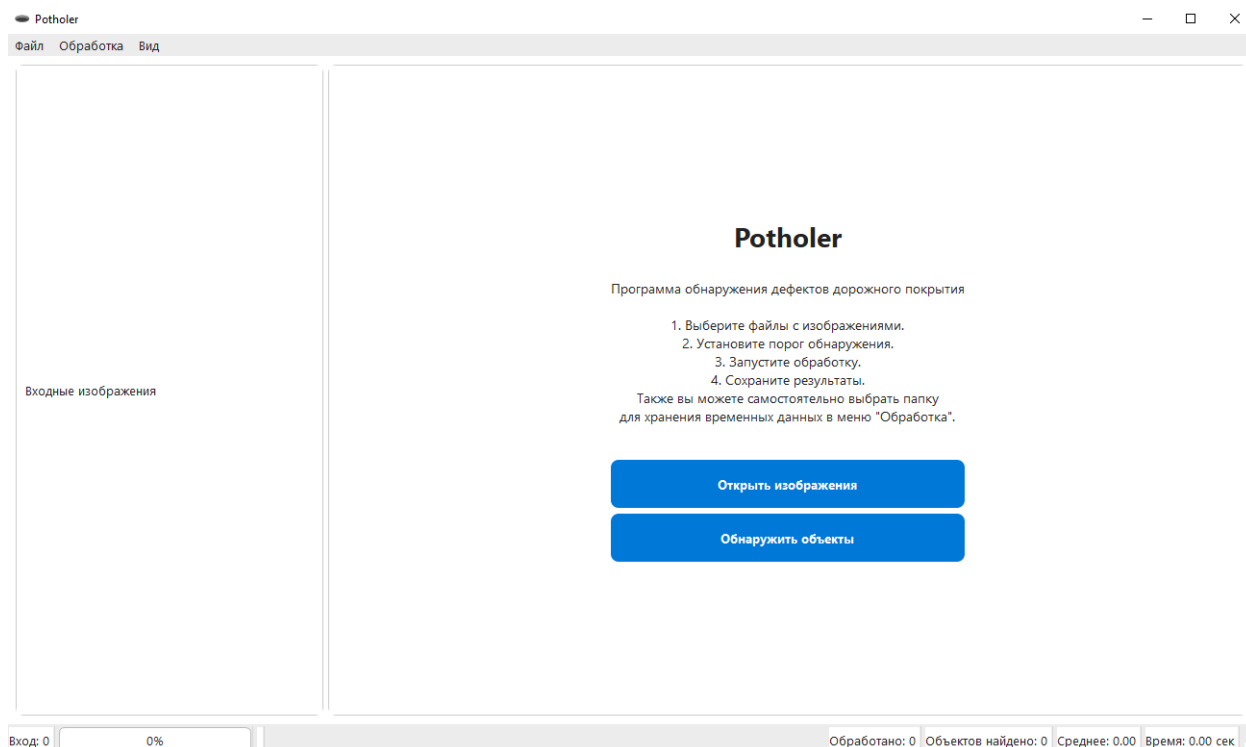


Рисунок 4.1 – Стартовый экран

В верхней части окна расположена строка меню, содержащая разделы «Файл», «Обработка» и «Вид». Раздел «Файл» предназначен для загрузки изображений, сохранения результатов обработки и открытия ранее сохранённых результатов. Раздел «Обработка» позволяет запускать процесс обнаружения дефектов дорожного покрытия, выбирать порог уверенности нейросетевой модели и управлять каталогом временных данных. Раздел «Вид» предоставляет средства управления отображением исходных изображений, сортировкой результатов и выбором темы оформления интерфейса.

В левой части главного окна отображается список загруженных изображений в виде миниатюр. Выбор миниатюры позволяет открыть соответствующее изображение в отдельном окне просмотра. Для уменьшения занимаемого пространства панель исходных изображений может быть скрыта пользователем.

Правая часть окна предназначена для отображения результатов обработки. После завершения обнаружения дефектов дорожного покрытия обработанные изображения отображаются в виде адаптивной сетки. Для каждого изображения дополнительно выводится количество обнаруженных дефек-

тов. При изменении размеров окна расположение элементов автоматически перестраивается.

В нижней части окна располагается строка состояния, содержащая информацию о количестве загруженных и обработанных изображений, общем числе обнаруженных дефектов, среднем количестве дефектов на изображение, времени выполнения обработки и текущем состоянии программы. Также отображается индикатор выполнения операций загрузки и обработки изображений.

Для удобства работы предусмотрено отдельное окно просмотра изображений, позволяющее просматривать исходные и обработанные изображения в увеличенном масштабе. Окно просмотра поддерживает переход к предыдущему и следующему изображениям без необходимости возврата к главному окну программы.

В программе реализованы светлая и тёмная темы оформления интерфейса. Выбранная пользователем тема автоматически сохраняется и восстанавливается при последующих запусках приложения.

4.9 Тестирование разработанной программной системы

Для проверки работоспособности программно-информационной системы обнаружения дефектов дорожного покрытия было выполнено комплексное тестирование, включающее модульное, интеграционное и системное тестирование. Проверка осуществлялась на наборах изображений дорожного покрытия с различным разрешением и содержанием дефектов.

4.9.1 Модульное тестирование компонентов системы

Модульное тестирование выполнялось с целью проверки корректности работы отдельных компонентов программной системы. Для автоматизации проверки использовался фреймворк `pytest`.

Для класса `Detector` проверялась корректность изменения порога уверенности обнаружения объектов, а также корректность выполнения мето-

да `process_images()`, включая формирование структуры возвращаемых данных после обработки изображений.

Для класса `FileManager` проверялась работа механизма формирования текстового отчёта, а также корректность загрузки и анализа ранее сохранённого файла отчёта.

Результаты модульного тестирования представлены на рисунке 4.2.

```
1 tests/test_detector.py::test_set_conf PASSED
2 tests/test_detector.py::test_process_images_basic PASSED
3 tests/test_file_manager.py::test_create_report PASSED
4 tests/test_report_parser.py::test_load_report PASSED
```

Рисунок 4.2 – Результат модульного тестирования

4.9.2 Системное тестирование

Системное тестирование выполнялось для проверки работоспособности программно-информационной системы обнаружения дефектов дорожного покрытия в целом. В ходе тестирования проверялись основные пользовательские сценарии работы.

Сценарий 1: загрузка изображений. Проверялась возможность загрузки одного или нескольких изображений с использованием диалога выбора файлов. После выбора файлов система должна отображать миниатюры изображений в левой части главного окна и обновлять информацию о количестве загруженных изображений.

В результате тестирования установлено, что изображения корректно загружаются в программу, отображаются в виде миниатюр.

Результат загрузки изображений представлен на рисунке 4.3.

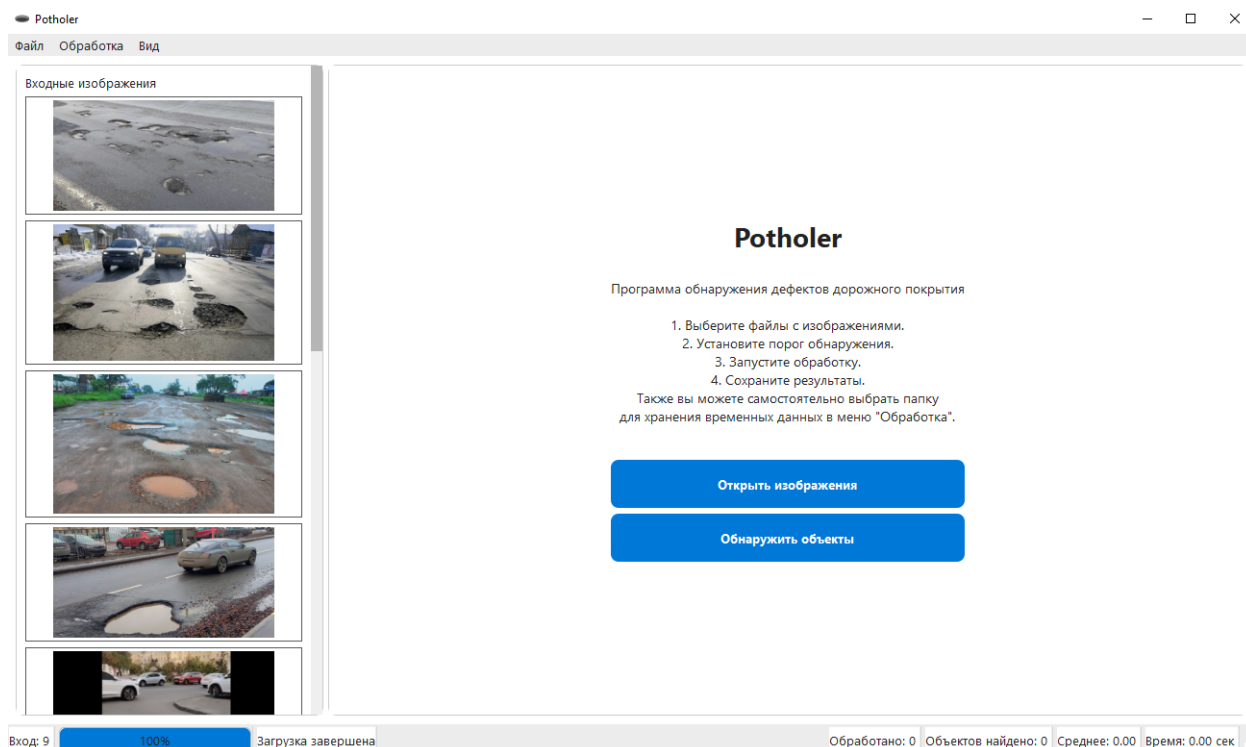


Рисунок 4.3 – Загрузка изображений в программу

Сценарий 2: настройка параметров обнаружения. Проверялась возможность изменения порога уверенности нейросетевой модели через меню программы. Для проверки последовательно выбирались все доступные значения порога обнаружения.

В ходе тестирования было установлено, что выбранное значение корректно передаётся модулю обнаружения объектов и используется при последующей обработке изображений.

Настройка параметров обнаружения представлена на рисунке 4.4.

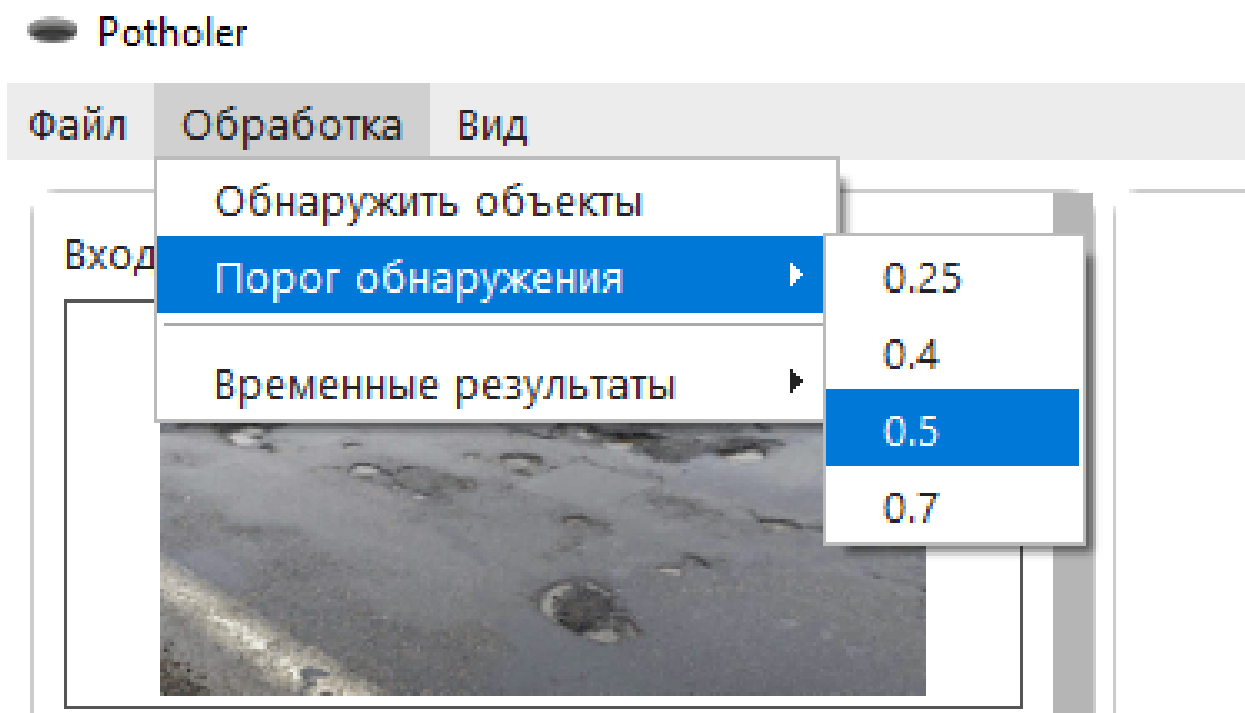


Рисунок 4.4 – Настройка порога обнаружения

Сценарий 3: анализ изображений. Проверялась работа алгоритма обнаружения дефектов дорожного покрытия на наборе тестовых изображений. В процессе тестирования выполнялась загрузка изображений различных форматов, их автоматическое масштабирование при превышении допустимого размера и выполнение анализа с использованием обученной модели YOLOv8n.

Также внимание уделялось корректности формирования ограничивающих рамок обнаруженных объектов, отображению подписи класса и значений уверенности модели. Дополнительно проверялись подсчёт общего количества обнаруженных дефектов, подсчёт количества дефектов на каждом изображении и обновление другой статистической информации в строке состояния.

В ходе тестирования подтверждена корректная работа алгоритма обнаружения дефектов дорожного покрытия, а также правильность формирования результатов обработки.

В результате тестирования было подтверждено корректное выполнение обнаружения дефектов дорожного покрытия, подсчёт найденных объектов и сохранение результатов обработки во временном каталоге.

Результат анализа изображений представлен на рисунке 4.5.

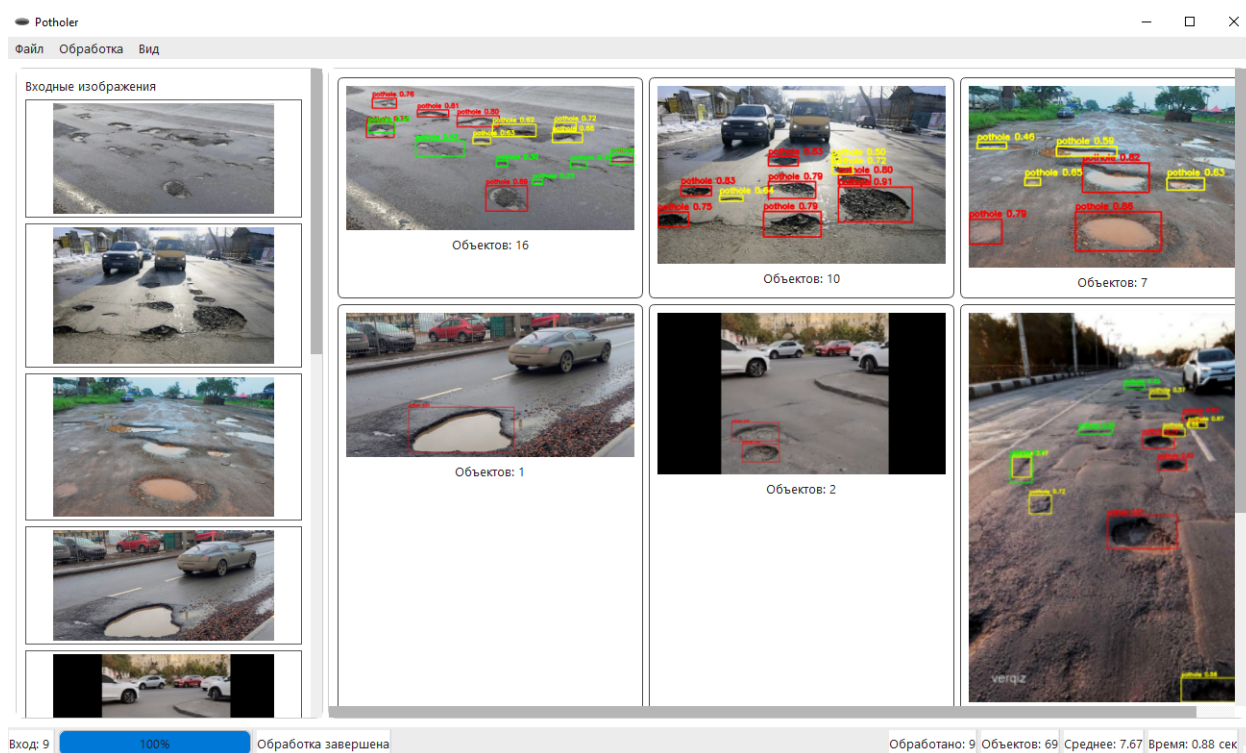


Рисунок 4.5 – Обнаружение дефектов дорожного покрытия

Сценарий 4: просмотр результатов обнаружения. Проверялась возможность просмотра результатов обработки в главном окне программы и в отдельном окне просмотра изображений, тестировалась навигация между изображениями с использованием кнопок перехода.

В ходе тестирования установлено, что результаты корректно отображаются в виде галереи, а просмотр изображений осуществляется без ошибок.

Просмотр результатов обнаружения представлен на рисунке 4.6.

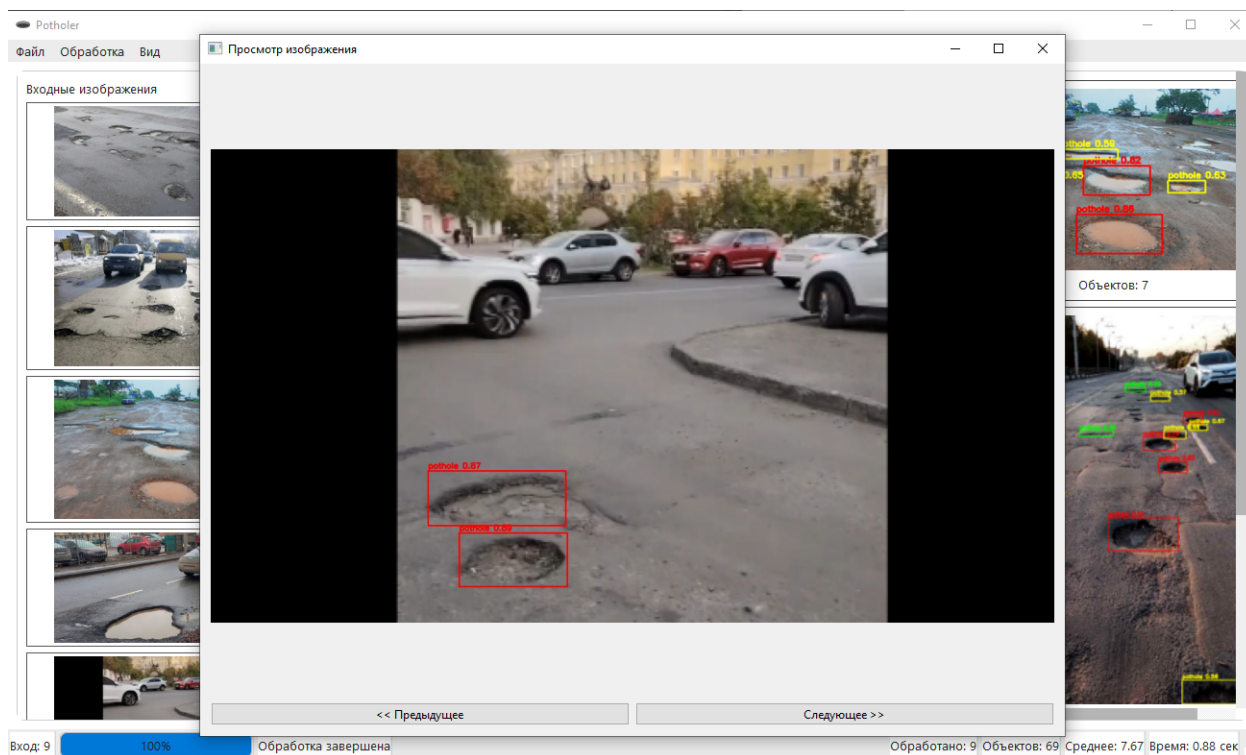


Рисунок 4.6 – Просмотр результатов обработки

Сценарий 5: сортировка результатов. Проверялась работа механизмов сортировки обработанных изображений по имени файла и по количеству обнаруженных объектов.

Результат сортировки представлен на рисунке 4.7.



Рисунок 4.7 – Сортировка результатов обработки

Сценарий 6: Сохранение результатов и формирование отчёта. Проверялась возможность сохранения обработанных изображений и автоматического формирования текстового отчёта о результатах анализа. В процессе тестирования контролировалась корректность копирования файлов в выбранный пользователем каталог, а также работа механизма обработки конфликтов имён существующих файлов.

Дополнительно проверялось формирование файла report.txt, содержащего количество обработанных изображений, число обнаруженных дефектов дорожного покрытия, среднее количество объектов на изображение, время обработки и статистику по каждому обработанному изображению.

В ходе тестирования было установлено, что программа корректно сохраняет изображения в выбранный каталог, формирует файл report.txt и записывает в него статистическую информацию о выполненной обработке.

Результат сохранения данных представлен на рисунке 4.8.

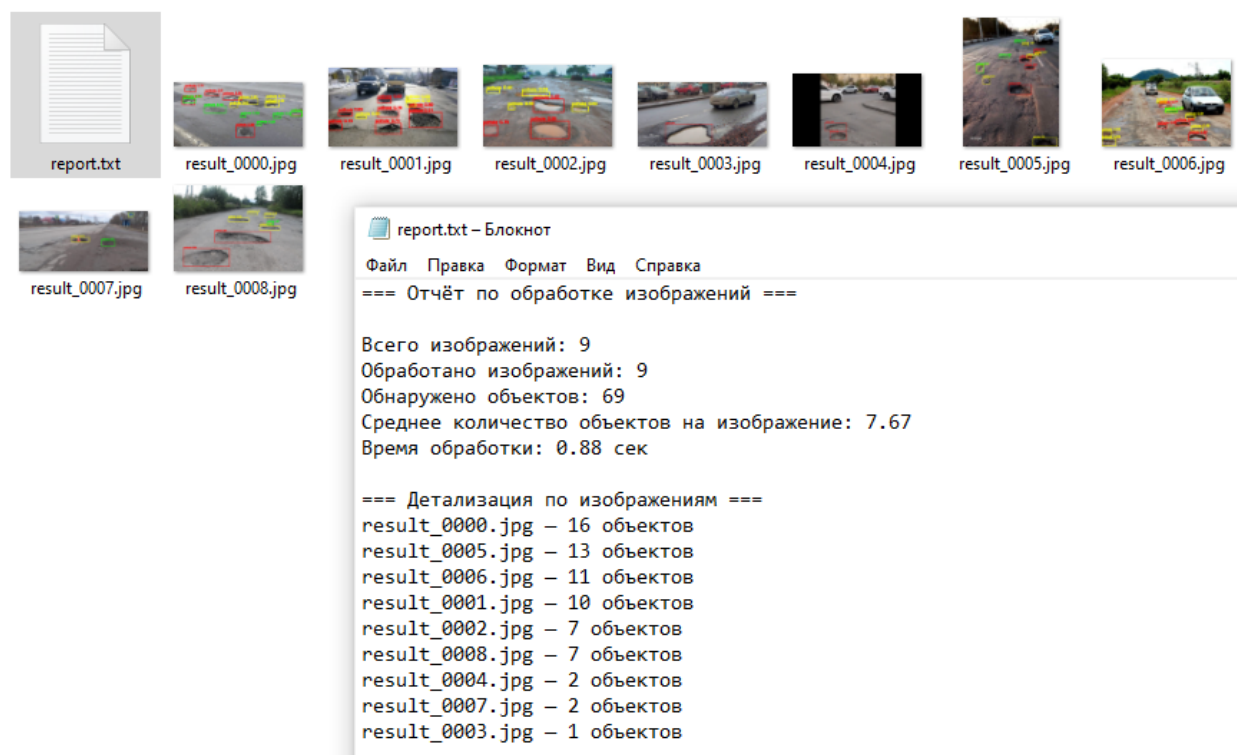


Рисунок 4.8 – Сохранение результатов обработки

Сценарий 7: загрузка ранее сохранённых результатов. Проверялась возможность открытия для просмотра ранее сохранённых результатов обработки. Контролировалось восстановление статистической информации о выполненной обработке, загрузка обработанных изображений и формирование галереи результатов. Для этого использовался каталог, содержащий обработанные изображения и файл отчёта.

В результате тестирования установлено, что программа корректно восстанавливает статистические данные, загружает список изображений и отображает результаты обработки.

Загрузка ранее сохранённых результатов представлена на рисунке 4.9.



Рисунок 4.9 – Загрузка ранее сохранённых результатов

Сценарий 8: очистка временных данных. Проверялась работа механизма очистки временного каталога, используемого для хранения промежуточных результатов обработки.

В ходе тестирования было подтверждено корректное удаление временных файлов и последующее создание пустого каталога для дальнейшей работы программы.

Очистка временных данных представлена на рисунках 4.10 и 4.11.

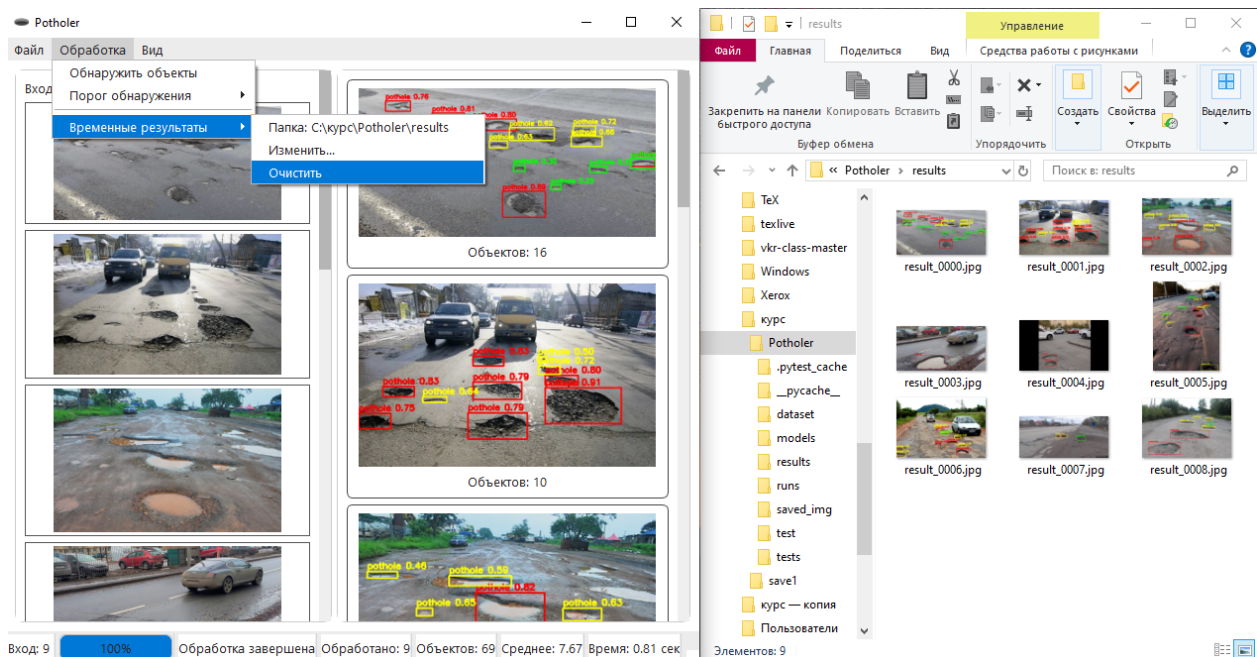


Рисунок 4.10 – До очистки временных данных

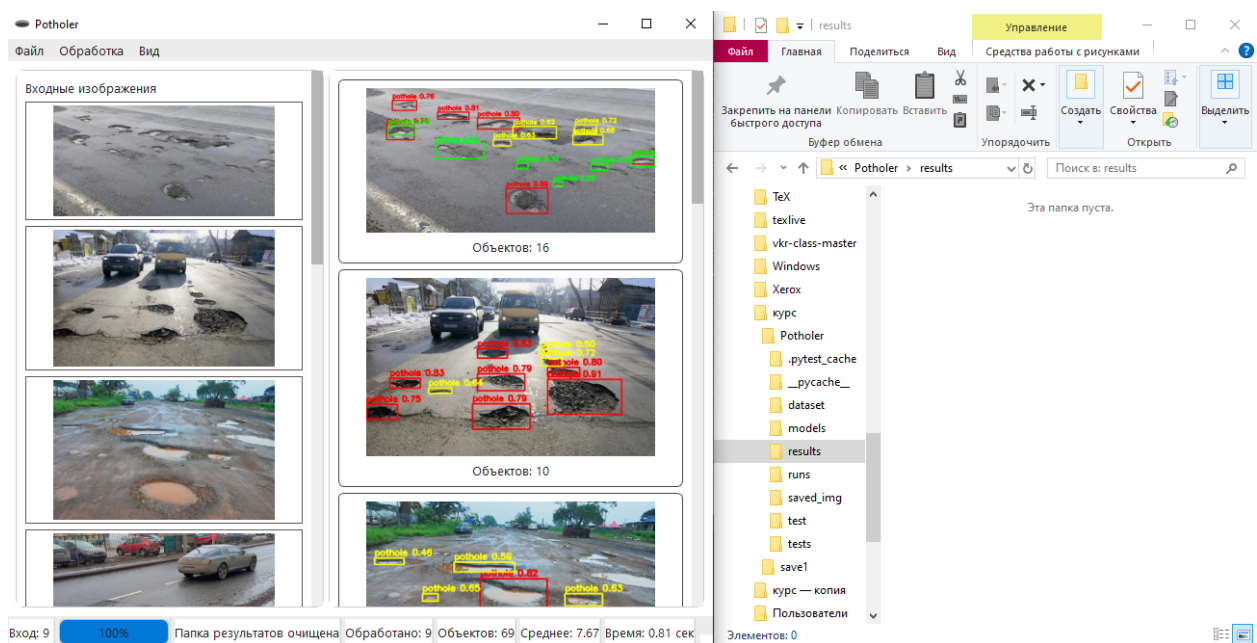


Рисунок 4.11 – После очистки временных данных

Сценарий 9: изменение директории сохранения временных файлов. Проверялась возможность выбора пользователем другой директории для сохранения временных файлов результатов обработки изображений.

В результате тестирования установлено, что выбранная пользователем директория корректно сохраняется в настройках приложения и используется при следующей обработке для временного хранения файлов.

Результаты изменения директории сохранения представлены на рисунке 4.12.

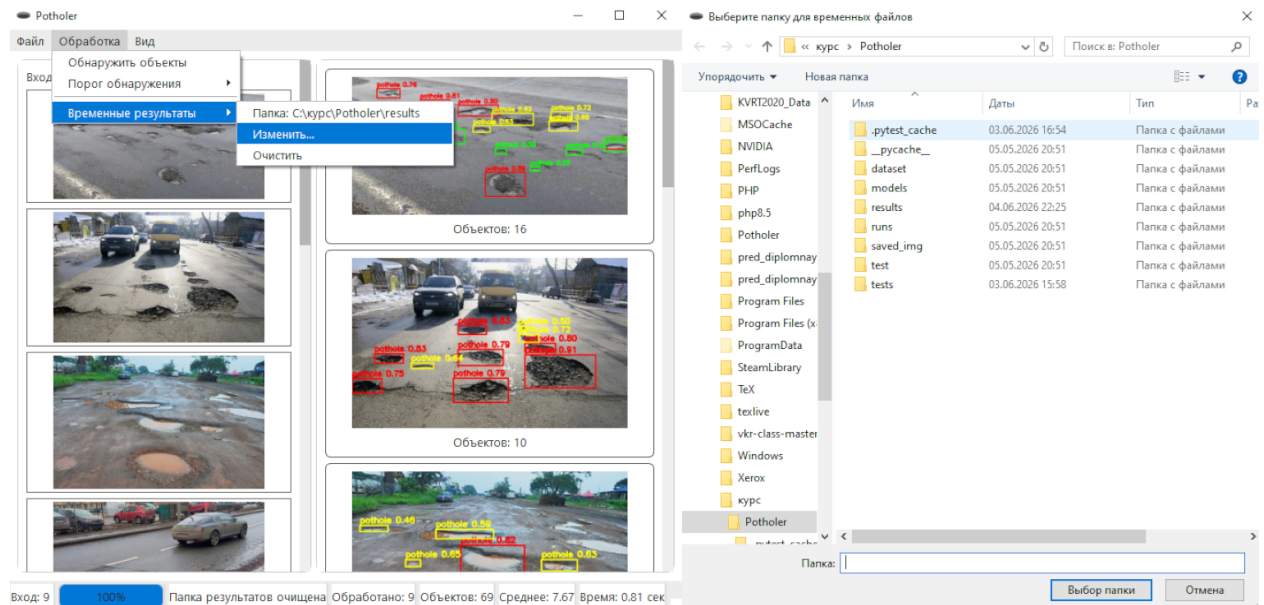


Рисунок 4.12 – Изменение директории сохранения файлов

Сценарий 10: переключение темы интерфейса. Проверялась возможность изменения оформления пользовательского интерфейса между светлой и тёмной темами.

Результаты тестирования показали, что тема оформления корректно изменяется без перезапуска программы, а выбранные настройки сохраняются и автоматически восстанавливаются при следующем запуске приложения.

Переключение темы интерфейса представлено на рисунке 4.13.

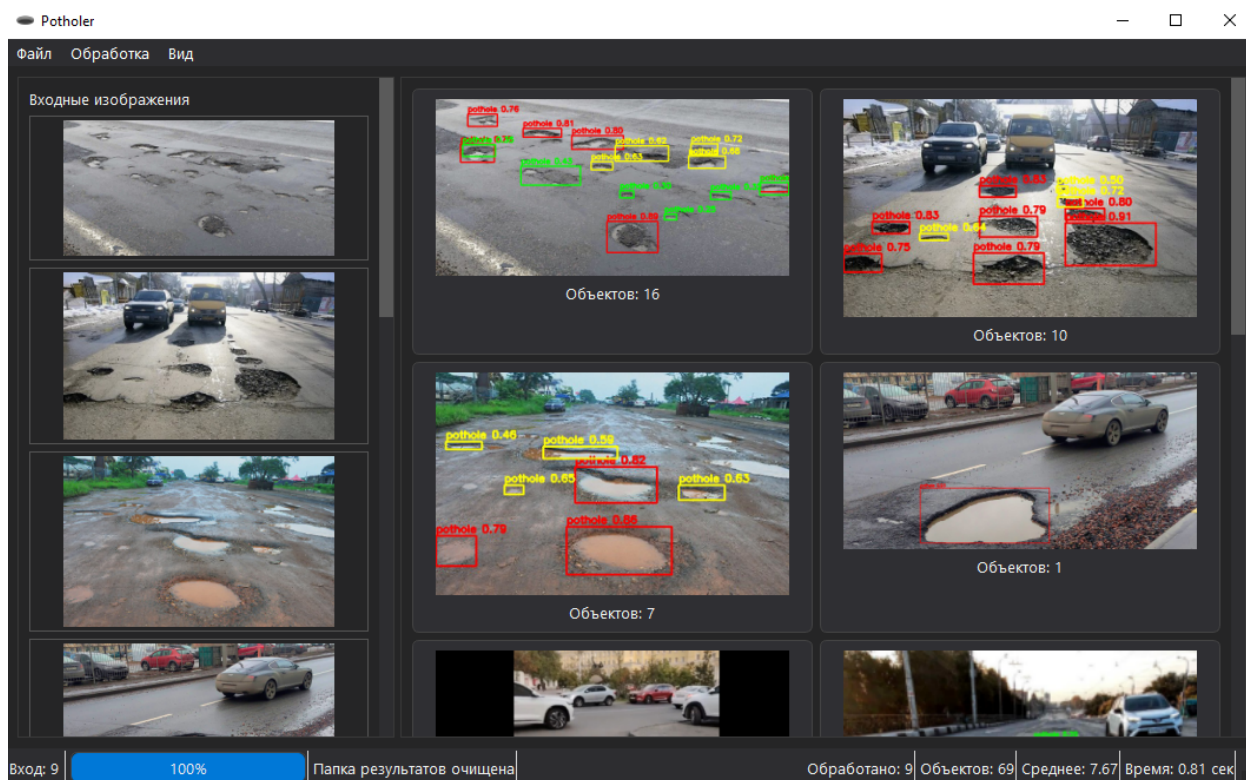


Рисунок 4.13 – Переключение темы интерфейса

Тестирование пользовательского интерфейса. Проверка интерфейсных компонентов выполнялась вручную.

Были протестированы:

- отображение миниатюр входных изображений;
- отображение миниатюр результатов обработки;
- работа прогресс-бара;
- навигация между изображениями;
- адаптивное изменение сетки результатов.

Критических нарушений в работе интерфейса не было выявлено.

ЗАКЛЮЧЕНИЕ

Разработанная программно-информационная система обнаружения дефектов дорожного покрытия на основе методов компьютерного зрения и нейросетевой обработки изображений обеспечивает автоматический анализ изображений дорожного покрытия, обнаружение дефектов с использованием обученной модели YOLOv8n, визуализацию результатов обработки, сохранение обработанных изображений и формирование текстового отчёта.

Основные результаты работы:

1. Проведён анализ предметной области и существующих методов обнаружения дефектов дорожного покрытия.
2. Изучены особенности архитектуры нейросетевых моделей семейства YOLO.
3. Выполнено проектирование программно-информационной системы и её пользовательского интерфейса.
4. Реализована программная система обнаружения дефектов дорожного покрытия на языке Python.
5. Выполнено обучение нейросетевой модели на специализированном наборе данных.
6. Реализованы механизмы обработки изображений, сохранения результатов и формирования отчётов.

Все требования, сформулированные в техническом задании, были реализованы в полном объёме. Поставленная цель разработки достигнута, все задачи, определённые в начале работы, успешно решены.

Результатом работы является программно-информационная система, позволяющая автоматизировать процесс обнаружения дефектов дорожного покрытия и сократить затраты времени на анализ изображений и сбор статистики по сравнению с ручными методами контроля.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Шакирьянов, Э. Д. Компьютерное зрение на Python. Первые шаги / Э. Д. Шакирьянов. – Москва : Лаборатория знаний, 2025. – 163 с. – ISBN 978-5-00101-944-2. – Текст : непосредственный.
2. Сацюк, А. В. Компьютерное зрение и нейронные сети. Практика / А. В. Сацюк. – Вологда : Инфра-Инженерия, 2025. – 364 с. – ISBN 978-5-9729-2706-7. – Текст : непосредственный.
3. Сацюк, А. В. Компьютерное зрение. Практика / А. В. Сацюк. – Вологда : Инфра-Инженерия, 2025. – 272 с. – ISBN 978-5-9729-2346-5. – Текст : непосредственный.
4. Цехановский, В. В., Бутырский, Е. Ю., Жукова, Н. А., Мельников, В. Б. Искусственные нейронные сети. Практикум / В. В. Цехановский, Е. Ю. Бутырский, Н. А. Жукова, В. Б. Мельников. – Москва : КноРус, 2026. – 381 с. – ISBN 978-5-406-15272-0. – Текст : непосредственный.
5. Ростовцев, В. С. Искусственные нейронные сети : учебник для вузов / В. С. Ростовцев. – Санкт-Петербург : Лань, 2025. – 216 с. – ISBN 978-5-507-50568-5. – Текст : непосредственный.
6. Николенко, С. И. Глубокое обучение: погружение в мир нейронных сетей / С. И. Николенко, А. Кадулин, Е. Архангельская. – Санкт-Петербург : Питер, 2025. – 476 с. – ISBN 9785446115372. – Текст : непосредственный.
7. Гуриков, С. Р. Программирование на языке Python / С. Р. Гуриков. – Москва : НИЦ ИНФРА-М, 2026. – 332 с. – ISBN 978-5-16-021562-4. – Текст : непосредственный.
8. Чернышев, С. А. Основы программирования на Python / С. А. Чернышев. – Москва : Издательство Юрайт, 2026. – 349 с. – ISBN 978-5-534-17139-6. – Текст : непосредственный.
9. Федоров, Д. Ю. Программирование на Python / Д. Ю. Федоров. – Москва : Издательство Юрайт, 2026. – 166 с. – ISBN 978-5-534-22181-7. – Текст : непосредственный.

10. Бильфельд, Н. В. Основы Python для будущих инженеров / Н. В. Бильфельд, А. В. Затонский. – Вологда : Инфра-Инженерия, 2026. – 468 с. – ISBN 978-5-9729-3137-8. – Текст : непосредственный.
11. Никитина, Т. П. Программирование. Основы Python для инженеров / Т. П. Никитина, Л. В. Королев. – Санкт-Петербург : Лань, 2026. – 156 с. – ISBN 978-5-507-51280-5. – Текст : непосредственный.
12. Дронов, В. А. Python 3 и PyQt 5. Разработка приложений / В. А. Дронов, Н. А. Прохоренок. – Санкт-Петербург : БХВ-Петербург, 2018. – 832 с. – ISBN 978-5-9775-3648-6. – Текст : непосредственный.
13. Машнин, Т. Создание настольных Python приложений с графическим интерфейсом пользователя / Т. Машнин. – [б. м.] : [б. и.], 2021. – 147 с. – ISBN 978-5-532-96908-7. – Текст : непосредственный.
14. Чаплыгин, А. А. Основы дипломного проектирования: учебное пособие для студентов направления подготовки 09.03.04 Программная инженерия, направленность (профиль) «Разработка программно-информационных систем» / А. А. Чаплыгин, В. В. Апальков, А. В. Малышев. – Курск : Юго-Зап. гос. ун-т, 2022. – 102 с. – ISBN 978-5-907407-65-7. – Текст : непосредственный.
15. Кальб, И. Объектно-ориентированное программирование с помощью Python / И. Кальб. – Москва : Эксмо, 2024. – 512 с. – ISBN 978-5-04-186627-3. – Текст : непосредственный.
16. Уилсон, Г. Разработка программного обеспечения на примерах с помощью Python / Г. Уилсон. – Москва : Бомбора, 2025. – 402 с. – ISBN 978-5-04-237641-2. – Текст : непосредственный.
17. Мэтиз, Э. Изучаем Python: программирование игр, визуализация данных, веб-приложения / Э. Мэтиз. – Санкт-Петербург : Питер, 2020. – 512 с. – ISBN 978-5-4461-1528-0. – Текст : непосредственный.
18. Левашов, П. Python с нуля / П. Левашов. – СПб. : Питер, 2024. – 448 с. – ISBN 978-5-4461-2145-8. – Текст : непосредственный.

19. Прохоренок, Н. А. Python 3. Самое необходимое / Н. А. Прохоренок, В. А. Дронов. – Санкт-Петербург : БХВ-Петербург, 2019. – 608 с. – ISBN 978-5-9775-3994-4. – Текст : непосредственный.
20. Белик, А. Г. Проектирование и архитектура программных систем : учебное пособие / А. Г. Белик, В. Н. Цыганенко. – Омск : ОмГТУ, 2016. – 96 с. – ISBN 978-5-8149-2258-8. – Текст : непосредственный.
21. Вигерс, К. Разработка требований к программному обеспечению: практические приёмы сбора требований и управления ими при разработке программных продуктов / К. Вигерс, Д. Битти. – Санкт-Петербург : БХВ, 2020. – 736 с. – ISBN 978-5-9775-3348-5. – Текст : непосредственный.
22. Юн, Ц. Рецепты Python. Коллекция лучших техник программирования / Ц. Юн. – Санкт-Петербург : Питер, 2024. – 512 с. – ISBN 978-5-4461-2156-4. – Текст : непосредственный.
23. Бхаргава, А. Грожаем алгоритмы / А. Бхаргава. – Санкт-Петербург : Питер, 2025. – 210 с. – ISBN 978-5-4461-4172-2. – Текст : непосредственный.
24. Python Documentation : официальный сайт. – URL: <https://www.python.org/doc/> (дата обращения: 16.05.2026). – Текст : электронный.
25. Шапиро, Л. Г. Компьютерное зрение : учебник для вузов / Л. Г. Шапиро, Д. В. Стокман. – Санкт-Петербург : БХВ-Петербург, 2020. – 756 с. – ISBN 978-5-9963-3003-4. – Текст : непосредственный.
26. Семакин, И. Г. Основы программирования : учебник / И. Г. Семакин. – Москва : Издательский центр «Академия», 2018. – 432 с. – ISBN 5-7695-2672-6. – Текст : непосредственный.
27. Демиденко, А. YOLO в действии: Обнаружение объектов / А. Демиденко. – [б. м.] : [б. и.], 2025. – URL: <https://www.litres.ru/book/artem-demidenko/yolo-v-deystvii-obnaruzhenie-obektov-71679058/> (дата обращения: 10.06.2026).

28. Матвеев, А. И. Цифровая обработка изображений в OpenCV. Практикум. Учебное пособие для вузов / А. И. Матвеев. – Санкт-Петербург : Лань, 2026. – 104 с. – ISBN 978-5-507-51334-5. – Текст : непосредственный.
29. Qingliang, Z. Qt 5 and OpenCV 4 Computer Vision Projects / Z. Qingliang. – Birmingham : Packt Publishing, 2019. – 350 с. – ISBN 9781789531831. – Текст : непосредственный.
30. Howse, J. Learning OpenCV 4 Computer Vision with Python 3 / J. Howse. – Birmingham : Packt Publishing, 2020. – 374 с. – ISBN 9781789530643. – Текст : непосредственный.
31. Willman, J. Beginning PyQt / J. Willman. – New York : Apress, 2022. – 568 с. – ISBN 9781484279984. – Текст : непосредственный.
32. Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин, пер. с англ. А. Кисилева. – Санкт-Петербург: Питер, 2018. – 351 с. – ISBN 978-5-4461-0772-8. – Текст: непосредственный.
33. Фаулер, М. UML. Основы / М. Фаулер, пер. с англ. А. Петухова. – 3-е изд. – Санкт-Петербург : Символ-Плюс, 2004. – 192 с. ISBN 978-5-93286-060-1. – Текст : непосредственный.
34. Шлее, М. Qt 5.10. Профессиональное программирование на C++ / М. Шлее. – Санкт-Петербург : БХВ-Петербург, 2018. – 1073 с. – ISBN 978-5-9775-3678-3. – Текст : непосредственный.
35. Букунова, О. В. Разработка приложений с графическим пользовательским интерфейсом на языке Python. Учебное пособие для СПО / О. В. Букунова, С. В. Букунов. – 3-е изд., стер. – Санкт-Петербург : Лань, 2026. – 89 с. – ISBN 978-5-507-54799-9. – Текст : непосредственный.
36. Qt for Python : официальный сайт. – URL: <https://doc.qt.io/qtforpython-6/> (дата обращения: 16.04.2026). – Текст : электронный.
37. Ultralytics YOLO : официальный сайт. – URL: <https://docs.ultralytics.com> (дата обращения: 10.04.2026). – Текст : электронный.

38. Куликова, И. В. Нейросети на Python. Основы ИИ и машинного обучения / И. В. Куликова. – Санкт-Петербург : Наука и техника, 2025. – 304 с. – ISBN 978-5-907592-55-1. – Текст : непосредственный.
39. Гудфеллоу, Я. Глубокое обучение / Я. Гудфеллоу, И. Бенджио, А. Курвилль, пер. с англ. А. А. Слинкина. – 2-е изд. – Москва : ДМК Пресс, 2018. – 652 с. – ISBN 978-5-97060-618-6. – Текст : непосредственный.
40. Шолле, Ф. Глубокое обучение на Python / Ф. Шолле, пер. с англ. А. Киселева. – 2-е межд. изд. – Санкт-Петербург : Питер, 2023. – 574 с. – ISBN 978-5-4461-0770-4. – Текст : непосредственный.
41. Аггарвал, Ч. Нейронные сети и глубокое обучение : учебный курс / Ч. Аггарвал. – Санкт-Петербург : Диалектика, 2020. – 754 с. – ISBN 9785907203013. – Текст : непосредственный.
42. Рашка, С. Python и машинное обучение. Машинное и глубокое обучение с использованием Python, scikit-learn и TensorFlow / С. Рашка, В. Мирджалили. – Санкт-Петербург : Диалектика, 2021. – 848 с. – ISBN 978-5-907203-57-0. – Текст : непосредственный.
43. Солем, Я. Э. Программирование компьютерного зрения на Python / Я. Э. Солем, пер. с англ. А. А. Слинкина. – Москва : ДМК Пресс, 2016. – 312 с. – ISBN 978-5-97060-200-3. – Текст : непосредственный.
44. Коул, А. Искусственный интеллект и компьютерное зрение. Реальные проекты на Python, Keras и TensorFlow / А. Коул, С. Ганджу, М. Казам, пер. с англ. А. Киселева. – Санкт-Петербург : Питер, 2023. – 624 с. – ISBN 978-5-4461-1840-3. – Текст : непосредственный.
45. Клетте, Р. Компьютерное зрение. Теория и алгоритмы / Р. Клетте, пер. с англ. А. А. Слинкина. – Москва : ДМК Пресс, 2019. – 506 с. – ISBN 978-5-97060-702-2. – Текст : непосредственный.
46. Корк, П. Машинное зрение и управление в робототехнике. Фундаментальные алгоритмы на Python / П. Корк, пер. с англ. А. Киселева. – Москва : ДМК Пресс, 2026. – 1106 с. – ISBN 978-5-93700-429-1. – Текст : непосредственный.

47. Золкин, А. Л. Методы идентификации и отслеживания объектов в системах дополненной реальности и компьютерного зрения. Учебное пособие для вузов / А. Л. Золкин. – Санкт-Петербург : Лань, 2025. – 172 с. – ISBN 978-5-507-53151-6. – Текст : непосредственный.

48. Селянкин, В. В. Компьютерное зрение. Анализ и обработка изображений. Учебное пособие для вузов / В. В. Селянкин. – 4-е изд., стер. – Санкт-Петербург : Лань, 2026. – 152 с. – ISBN 978-5-507-51201-0. – Текст : непосредственный.

49. Fitzpatrick, M. Create GUI Applications with Python & Qt / M. Fitzpatrick. – [б. м.] : [б. и.], 2025. – 722 с. – ISBN 9798287562120. – Текст : непосредственный.

50. Jaganmohan, G. PySide GUI Application Development / G. Jaganmohan. – Birmingham : Packt Publishing, 2016. – 239 с. – ISBN 9781785280368. – Текст : непосредственный.

51. Chandrakar, S. Python GUI with PyQt / S. Chandrakar – New Delhi : BPB Publications, 2023. – 440 с. – ISBN 9789355515575. – Текст : непосредственный.

ПРИЛОЖЕНИЕ А

Представление графического материала

Графический материал, выполненный на отдельных листах, изображен на рисунках А.1–А.6.

Рисунок А.1 – Сведения о ВКРБ

Цель и задачи разработки

Цель настоящей работы – создание программно-информационной системы обнаружения дефектов дорожного покрытия на основе нейросетевых методов анализа изображений.

Для достижения поставленной цели необходимо решить следующие задачи:

- анализ предметной области и существующих методов обнаружения дефектов дорожного покрытия;
- изучение особенностей архитектур нейросетевых моделей обнаружения объектов;
- проектирование программной системы;
- обучение нейросетевой модели;
- проектирование пользовательского интерфейса приложения;
- реализация обработки изображений с использованием нейросетевой модели;
- реализация механизмов сохранения результатов и формирования отчёта.

ВКРБ 2206127.09.03.04.26.003			
Имя	Фамилия	Имя	Фамилия
Имя	Фамилия	Имя	Фамилия
Имя	Фамилия	Имя	Фамилия
Цель и задачи разработки		Лист	Из всего
Выполнен идентификацией		Лист 2	Лист 4
работы по созданию		03.09.2025	

Рисунок А.2 – Цель и задачи разработки



Рисунок А.3 – Диаграмма прецедентов



Рисунок А.4 – Диаграмма компонентов

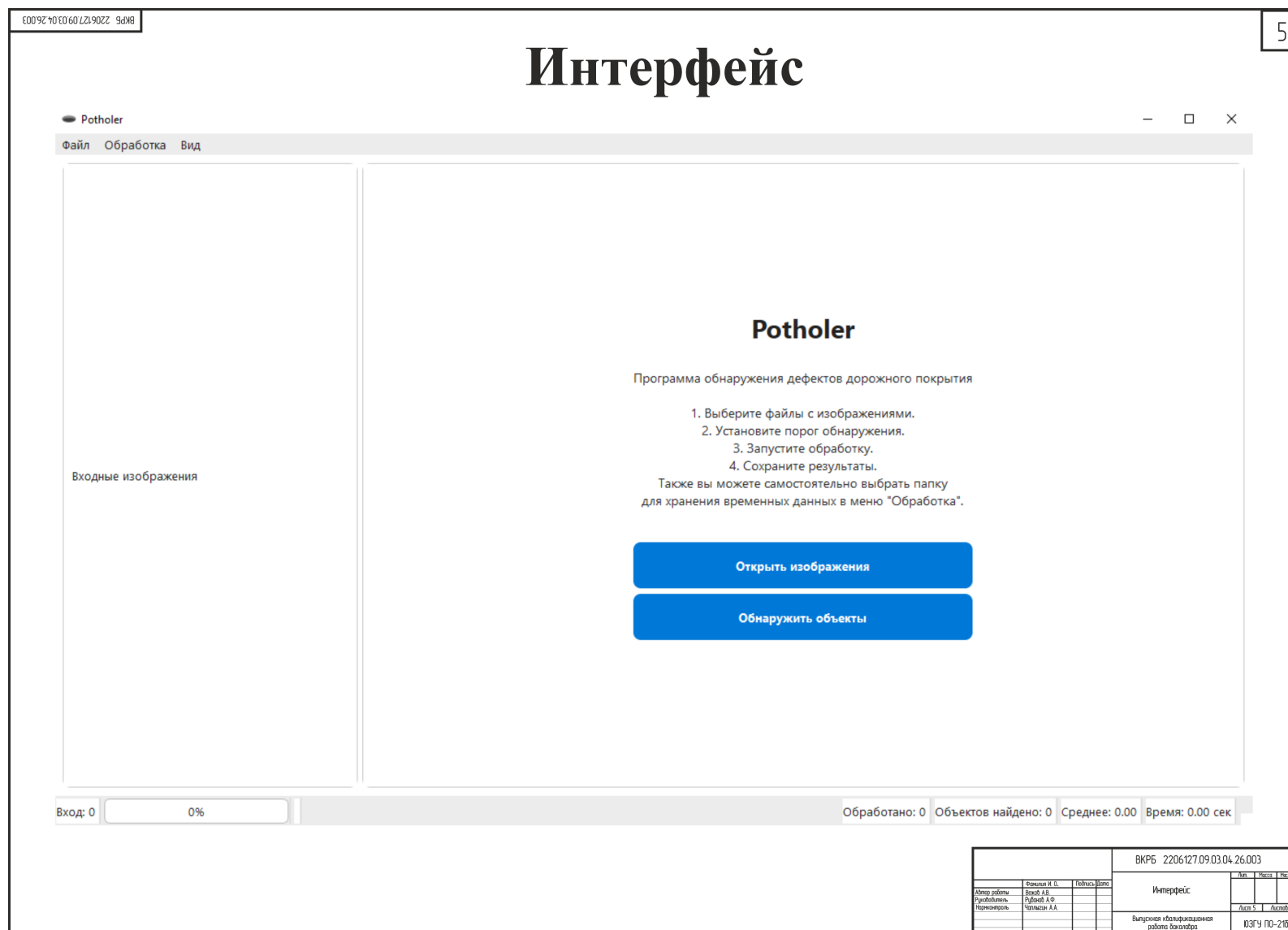


Рисунок А.5 – Интерфейс

Рисунок А.6 – Заключение

ПРИЛОЖЕНИЕ Б

Фрагменты исходного кода программы

detector.py

```
1 import os
2 import shutil
3 import numpy as np
4 import cv2
5
6 from PySide6.QtGui import QImageReader, QImage
7 from PySide6.QtCore import QSize
8
9 class Detector:
10     def __init__(self, model, conf_threshold=0.25, max_dim=1920):
11         self.model = model
12         self.conf_threshold = conf_threshold
13         self.max_dim = max_dim
14
15     def set_conf(self, value):
16         self.conf_threshold = value
17
18     def process_images(self, images, results_dir, progress_callback=None,
19                       stop_flag=None):
20         processed_paths = []
21         per_image_detections = []
22         total_detections = 0
23
24         if os.path.exists(results_dir):
25             shutil.rmtree(results_dir)
26             os.makedirs(results_dir)
27
28         for idx, path in enumerate(images):
29
30             if stop_flag and stop_flag():
31                 break
32
33             reader = QImageReader(path)
34             reader.setAutoTransform(True)
35             size = reader.size()
36
37             if not size.isValid():
38                 continue
39
40             if max(size.width(), size.height()) > self.max_dim:
41                 scale = self.max_dim / max(size.width(), size.height())
42                 reader.setScaledSize(QSize(
43                     int(size.width() * scale),
44                     int(size.height() * scale)
45                 ))
46
47             image = reader.read()
48             if image.isNull():
49                 continue
```

```

49
50 image = image.convertToFormat(QImage.Format_RGB888)
51
52 w, h = image.width(), image.height()
53 ptr = image.bits()
54 arr = np.frombuffer(ptr, dtype=np.uint8).copy()
55
56 img = arr.reshape(h, image.bytesPerLine(), 1)[: , :w*3].reshape(h,
57                               w, 3)
58 img = np.ascontiguousarray(img)
59
60 image_detections = 0
61
62 results = self.model(img, conf=self.conf_threshold, verbose=False
63 )
64
65 for r in results:
66     if r.bboxes is None:
67         continue
68
69     for box in r.bboxes:
70         total_detections += 1
71         image_detections += 1
72
73         x1, y1, x2, y2 = map(int, box.xyxy[0])
74         cls = int(box.cls[0])
75         conf = float(box.conf[0])
76
77         label = f"{self.model.names[cls]} {conf:.2f}"
78
79         if conf > 0.75:
80             color = (255, 0, 0)
81         elif conf > 0.45:
82             color = (255, 255, 0)
83         else:
84             color = (0, 255, 0)
85
86         cv2.rectangle(img, (x1, y1), (x2, y2), color, 2)
87         cv2.putText(img, label, (x1, y1 - 5),
88                     cv2.FONT_HERSHEY_SIMPLEX, 0.5, color, 2)
89
90 per_image_detections.append(image_detections)
91
92 save_path = os.path.join(results_dir, f"result_{idx:04d}.jpg")
93
94 cv2.imwrite(
95     save_path,
96     cv2.cvtColor(img, cv2.COLOR_RGB2BGR),
97     [int(cv2.IMWRITE_JPEG_QUALITY), 90]
98 )
99
100 processed_paths.append(save_path)
101
102 if progress_callback:

```

```

101         progress_callback(idx + 1, len(images), total_detections)
102
103     return processed_paths, per_image_detections, total_detections

```

main_window.py

```

1  import sys
2  import faulthandler
3  import signal
4  faulthandler.enable(True)
5
6  import numpy as np
7  import cv2
8  import os
9  import shutil
10 import time
11 from ultralytics import YOLO
12
13 from PySide6.QtWidgets import (
14     QApplication, QMainWindow, QWidget,
15     QHBoxLayout, QVBoxLayout, QGridLayout,
16     QScrollArea, QLabel, QFileDialog,
17     QPushButton, QSplitter, QStatusBar,
18     QProgressBar, QMessageBox
19 )
20 from PySide6.QtGui import QPixmap, QImage, QImageReader, QIcon
21 from PySide6.QtCore import Qt, QSize, QTimer, QSettings
22
23 from viewer import ImageViewer
24 from detector import Detector
25 from theme_manager import ThemeManager
26 from file_manager import FileManager
27 from gallery_manager import GalleryManager
28
29 THUMB_SIZE = 140
30
31
32 class MainWindow(QMainWindow):
33
34     def init_window(self):
35         self.setWindowIcon(QIcon("icon.png"))
36         self.setWindowTitle("Potholer")
37         self.resize(1300, 750)
38
39     def init_settings(self):
40         self.settings = QSettings("Potholer", "Detector")
41         self.current_theme = self.settings.value("theme", "dark")
42
43         self.results_dir = self.settings.value("results_dir")
44
45         if not self.results_dir:
46             self.results_dir = os.path.normpath(
47                 os.path.abspath("results")
48             )
49

```

```

50 def init_managers(self):
51     self.theme_manager = ThemeManager(self)
52     self.file_manager = FileManager(self)
53     self.gallery = GalleryManager(self)
54
55 def init_state(self):
56     self.images = []
57
58     self.result_pixmap = []
59     self.processed_paths = []
60     self.per_image_detections = []
61     self.total_detections = 0
62
63     self.stop_requested = False
64     self.loading_images = False
65     self.processing = False
66     self.show_input_images = True
67     self.results_saved = False
68
69     self.welcome_widget = None
70
71 def init_detector(self):
72     self.detector = Detector(
73         YOLO("runs/detect/train5/weights/best.pt"),
74         conf_threshold=0.25
75     )
76
77 def init_menu(self):
78     menu_bar = self.menuBar()
79     file_menu = menu_bar.addMenu("Файл")
80     open_action = file_menu.addAction("Открыть изображения")
81     open_action.triggered.connect(self.file_manager.open_images)
82     save_action = file_menu.addAction("Сохранить результаты")
83     save_action.triggered.connect(self.save_results)
84     open_processed_action = file_menu.addAction("Открыть обработанные
85         результаты")
86     open_processed_action.triggered.connect(self.open_processed_results)
87     exit_action = file_menu.addAction("Выход")
88     exit_action.triggered.connect(self.close)
89
90     process_menu = menu_bar.addMenu("Обработка")
91     self.detect_action = process_menu.addAction("Обнаружить объекты")
92     self.detect_action.triggered.connect(self.detect_objects)
93     conf_menu = process_menu.addMenu("Порог обнаружения")
94     process_menu.addSeparator()
95     results_menu = process_menu.addMenu("Временные результаты")
96     self.current_dir_action = results_menu.addAction("Папка: не выбрана")
97     self.current_dir_action.setEnabled(False)
98     change_dir_action = results_menu.addAction("Изменить...")
99     change_dir_action.triggered.connect(self.file_manager.
100         select_results_folder)
101     clear_dir_action = results_menu.addAction("Очистить")
102     clear_dir_action.triggered.connect(self.file_manager.
103         clear_results_folder)

```

```

101
102     for conf in [0.25, 0.40, 0.50, 0.70]:
103         action = conf_menu.addAction(str(conf))
104         action.triggered.connect(lambda checked, c=conf: self.
            set_conf_threshold(c))
105
106     view_menu = menu_bar.addMenu("Вид")
107     self.toggle_input_action = view_menu.addAction("Показывать входные
        изображения")
108     self.toggle_input_action.setCheckable(True)
109     self.toggle_input_action.setChecked(True)
110     self.toggle_input_action.triggered.connect(self.toggle_input_images)
111     sort_menu = view_menu.addMenu("Сортировка результатов")
112     sort_by_name = sort_menu.addAction("По имени")
113     sort_by_count = sort_menu.addAction("По числу объектов")
114     sort_by_name.triggered.connect(lambda: self.sort_results("name"))
115     sort_by_count.triggered.connect(lambda: self.sort_results("count"))
116     theme_menu = view_menu.addMenu("Тема")
117     dark_theme_action = theme_menu.addAction("Тёмная")
118     light_theme_action = theme_menu.addAction("Светлая")
119     dark_theme_action.triggered.connect(lambda: self.theme_manager.
        set_theme("dark"))
120     light_theme_action.triggered.connect(lambda: self.theme_manager.
        set_theme("light"))
121
122
123     def init_wid(self):
124         #центр
125         central_widget = QWidget()
126         self.setCentralWidget(central_widget)
127         main_layout = QHBoxLayout(central_widget)
128
129         #лево
130         self.left_scroll = QScrollArea()
131         self.left_scroll.setWidgetResizable(True)
132         self.left_container = QWidget()
133         self.left_scroll.setWidget(self.left_container)
134         self.left_layout = QVBoxLayout(self.left_container)
135         self.left_layout.addWidget(QLabel("Входные изображения"))
136         self.left_scroll.setFixedWidth(320)
137
138
139         #право
140         self.right_scroll = QScrollArea()
141         self.right_scroll.setWidgetResizable(True)
142         self.right_container = QWidget()
143         self.right_scroll.setWidget(self.right_container)
144         self.right_layout = QVBoxLayout(self.right_container)
145         self.result_grid = QGridLayout()
146         self.right_layout.addLayout(self.result_grid)
147         #self.right_layout.addStretch()
148
149         self.resize_timer = QTimer()
150         self.resize_timer.setSingleShot(True)

```

```

151         self.resize_timer.timeout.connect(self.gallery.update_result_grid)
152
153         #сплиттер
154         splitter = QSplitter(Qt.Horizontal)
155         splitter.addWidget(self.left_scroll)
156         splitter.addWidget(self.right_scroll)
157         splitter.setSizes([300, 1000])
158         main_layout.addWidget(splitter)
159
160     def init_statusbar(self):
161         self.status = QStatusBar()
162         self.setStatusBar(self.status)
163
164         self.input_counter = QLabel("Вход: 0 ")
165         self.processed_counter = QLabel("Обработано: 0 ")
166         self.detection_counter = QLabel("Объектов найдено: 0 ")
167         self.avg_counter = QLabel("Среднее: 0.00 ")
168         self.time_counter = QLabel("Время: 0.00 сек ")
169
170         self.progress_bar = QProgressBar()
171         self.progress_bar.setMaximumWidth(200)
172         self.progress_bar.setValue(0)
173
174         self.status_text = QLabel("") #сообщ о состоянии
175
176         self.status.addWidget(self.input_counter)
177         self.status.addWidget(self.progress_bar)
178         self.status.addWidget(self.status_text)
179
180         self.status.addPermanentWidget(self.processed_counter)
181         self.status.addPermanentWidget(self.detection_counter)
182         self.status.addPermanentWidget(self.avg_counter)
183         self.status.addPermanentWidget(self.time_counter)
184
185
186     def __init__(self):
187         super().__init__()
188
189
190         self.init_window()
191         self.init_settings()
192         self.init_state()
193         self.init_managers()
194         self.init_detector()
195         self.init_menu()
196         self.init_wid()
197         self.init_statusbar()
198
199         self.show_welcome_screen()
200         self.update_results_dir_display()
201         self.theme_manager.set_theme(self.current_theme)
202
203
204     def update_progress(self, current, total, text=None):

```

```

205         self.progress_bar.setMaximum(total)
206         self.progress_bar.setValue(current)
207
208         if text:
209             self.status_text.setText(text)
210
211         QApplication.processEvents()
212
213     #приветственное
214     def show_welcome_screen(self):
215         self.clear_welcome_screen()
216
217         self.welcome_widget = QWidget()
218         layout = QVBoxLayout(self.welcome_widget)
219         layout.setAlignment(Qt.AlignCenter)
220
221         title = QLabel("Potholer")
222         title.setAlignment(Qt.AlignCenter)
223         title.setStyleSheet("""
224             font-size: 28px;
225             font-weight: bold;
226             """)
227
228         info = QLabel(
229             "Программа обнаружения дефектов дорожного покрытия\n\n"
230             "1. Выберите файлы с изображениями.\n"
231             "2. Установите порог обнаружения.\n"
232             "3. Запустите обработку.\n"
233             "4. Сохраните результаты.\n"
234
235             "Также вы можете самостоятельно выбрать папку\ндля хранения
236             временных данных в меню \"Обработка\"."
237         )
238         info.setAlignment(Qt.AlignCenter)
239         info.setStyleSheet("font-size: 14px;")
240
241         open_btn = QPushButton("Открыть изображения")
242         detect_btn = QPushButton("Обнаружить объекты")
243
244         open_btn.setMinimumHeight(50)
245         detect_btn.setMinimumHeight(50)
246
247         open_btn.clicked.connect(self.file_manager.open_images)
248         detect_btn.clicked.connect(self.detect_objects)
249
250         layout.addWidget(title)
251         layout.addSpacing(20)
252         layout.addWidget(info)
253         layout.addSpacing(30)
254         layout.addWidget(open_btn)
255         layout.addWidget(detect_btn)
256
257         self.right_layout.addStretch()
258         self.right_layout.addWidget(self.welcome_widget)

```

```

258         self.right_layout.addStretch()
259
260     def clear_welcome_screen(self):
261         if self.welcome_widget:
262             self.welcome_widget.setParent(None)
263             self.welcome_widget = None
264
265
266
267     def update_results_dir_display(self):
268         path = self.results_dir
269
270         if not path:
271             text = "Папка не выбрана"
272         else:
273             short = path
274             if len(short) > 45:
275                 short = "..." + short[-42:]
276             text = f"Папка: {short}"
277
278         self.current_dir_action.setText(text)
279
280
281     def set_images(self, files):
282         self.images = files
283         self.input_counter.setText(f"Вход: {len(files)} ")
284
285         if self.show_input_images:
286             self.gallery.show_thumbnails()
287         else:
288             self.status_text.setText("Загрузка изображений...")
289
290             for idx, _ in enumerate(files):
291                 self.update_progress(idx + 1, len(files))
292
293             self.status_text.setText("Загрузка завершена")
294
295     def save_results(self):
296         result = self.file_manager.save_results()
297
298         if result is False:
299             self.status_text.setText("Сохранение отменено")
300             return
301
302         if result is None:
303             self.status_text.setText("Ошибка при сохранении отчёта")
304             return
305
306         self.results_saved = True
307         self.status_text.setText(
308             f"Сохранено: {result} изображений + отчёт"
309         )
310
311

```

```

312
313 def open_processed_results(self):
314
315     data = self.file_manager.open_processed_results()
316
317     if not data:
318         self.status_text.setText("Ошибка загрузки результатов")
319         return
320
321     self.clear_welcome_screen()
322
323     self.processed_paths = data["paths"]
324     self.per_image_detections = data["detections"]
325
326     self.processed_counter.setText(f"Обработано: {len(self.
        processed_paths)}")
327
328     self.detection_counter.setText(f"Объектов: {data['total']}")
329
330     self.avg_counter.setText(f"Среднее: {data['avg']}")
331     self.time_counter.setText(f"Время: {data['time']}")
332
333     self.total_detections = data["total"]
334
335     self.gallery.display_results()
336
337     self.status_text.setText("Обработанные результаты загружены")
338
339
340 def toggle_input_images(self, checked):
341     self.show_input_images = checked
342     self.gallery.toggle_input_visibility(checked)
343
344
345 def set_conf_threshold(self, value):
346     self.detector.set_conf(value)
347     self.status_text.setText(f"Порог обнаружения: {value}")
348
349
350 def sort_results(self, mode):
351     if not self.processed_paths:
352         return
353
354     combined = list(zip(
355         self.processed_paths,
356         self.result_pixmap,
357         self.per_image_detections
358     ))
359
360     if mode == "count":
361         combined.sort(key=lambda x: x[2], reverse=True)
362     else:
363         combined.sort(key=lambda x: os.path.basename(x[0]))
364

```

```

365         self.processed_paths, self.result_pixmap, self.per_image_detections
           = zip(*combined)
366
367         self.processed_paths = list(self.processed_paths)
368         self.result_pixmap = list(self.result_pixmap)
369         self.per_image_detections = list(self.per_image_detections)
370
371         self.gallery.update_result_grid()
372
373
374     def resizeEvent(self, event):
375         if self.result_pixmap:
376             self.resize_timer.start(200)
377
378
379
380     def select_image(self, index):
381         self.open_viewer(index)
382
383     def open_viewer(self, index):
384         self.viewer = ImageViewer(self.images, index)
385         self.viewer.show()
386
387     def open_processed_viewer(self, index):
388         self.proc_viewer = ImageViewer(self.processed_paths, index)
389         self.proc_viewer.show()
390
391
392     def detect_objects(self):
393
394         self.clear_welcome_screen()
395
396         if not self.images or self.processing:
397             return
398
399         self.results_saved = False
400         self.processing = True
401         self.detect_action.setEnabled(False)
402
403         self.processing_start_time = time.time()
404
405         self.processed_paths = []
406         self.result_pixmap = []
407         self.per_image_detections = []
408         self.total_detections = 0
409
410         self.detection_counter.setText("Объектов: 0")
411         self.avg_counter.setText("Среднее: 0.00")
412         self.time_counter.setText("Время: 0.00 сек")
413
414         QApplication.setOverrideCursor(Qt.WaitCursor)
415
416         self.status_text.setText("Обработка изображений...")
417

```

```

418     def progress(current, total, total_det):
419         self.progress_bar.setMaximum(total)
420         self.progress_bar.setValue(current)
421
422         self.processed_counter.setText(f"Обработано: {current}")
423         self.detection_counter.setText(f"Объектов: {total_det}")
424
425         QApplication.processEvents()
426
427     self.processed_paths, self.per_image_detections, self.
total_detections = \
428         self.detector.process_images(
429             self.images,
430             self.results_dir,
431             progress_callback=progress,
432             stop_flag=lambda: self.stop_requested
433         )
434
435     QApplication.restoreOverrideCursor()
436
437     self.processing = False
438     self.detect_action.setEnabled(True)
439
440     self.processing_time = time.time() - self.processing_start_time
441
442     avg = (
443         self.total_detections / len(self.processed_paths)
444         if self.processed_paths else 0
445     )
446
447     self.avg_counter.setText(f"Среднее: {avg:.2f}")
448     self.time_counter.setText(f"Время: {self.processing_time:.2f} сек")
449
450     QTimer.singleShot(0, self.gallery.display_results)
451     self.status_text.setText("Обработка завершена")
452
453
454
455     def closeEvent(self, event):
456         #если идёт
457         if self.loading_images or self.processing:
458
459             if self.loading_images:
460                 text = "Сейчас выполняется загрузка изображений.\nВы уверены,
что хотите выйти?"
461
462             else:
463                 text = "Сейчас выполняется обработка изображений.\nВы уверены
, что хотите выйти?"
464
465             msg = QMessageBox(self)
466             msg.setWindowTitle("Подтверждение выхода")
467             msg.setText(text)
468
469             yes_btn = msg.addButton("Выйти", QMessageBox.AcceptRole)

```

```

469         no_btn = msg.addButton("Отмена", QMessageBox.RejectRole)
470
471     msg.exec()
472
473     if msg.clickedButton() == yes_btn:
474         self.stop_requested = True
475         event.accept()
476     else:
477         event.ignore()
478
479     return
480
481     #есть несохранённые
482     if self.processed_paths and not self.results_saved:
483
484         msg = QMessageBox(self)
485         msg.setWindowTitle("Несохранённые результаты")
486         msg.setText(
487             "Результаты обработки не сохранены.\n"
488             "Сохранить перед выходом?"
489         )
490
491         save_btn = msg.addButton("Сохранить", QMessageBox.AcceptRole)
492         exit_btn = msg.addButton("Выйти без сохранения", QMessageBox.
            DestructiveRole)
493         cancel_btn = msg.addButton("Отмена", QMessageBox.RejectRole)
494
495         msg.exec()
496
497         clicked = msg.clickedButton()
498
499         if clicked == save_btn:
500             self.save_results()
501
502             #отмена
503             if self.results_saved:
504                 event.accept()
505             else:
506                 event.ignore()
507
508         elif clicked == exit_btn:
509             event.accept()
510
511         else:
512             event.ignore()
513
514     return
515
516     #обычный
517     event.accept()

```

Автор ВКР

(подпись, дата)

А. В. Вожов

Руководитель ВКР

(подпись, дата)

А. Ф. Рубанов

Нормоконтроль

(подпись, дата)

А. А. Чаплыгин

Место для диска